



BASE24[®]

IEP Configuration Manual

© 2000 by ACI Worldwide Inc. All rights reserved.

All information contained in this document is confidential and proprietary to ACI Worldwide Inc. This material is a trade secret and its confidentiality is strictly maintained. Use of any copyright notice does not imply unrestricted or public access to these materials. No part of this document may be photocopied, electronically transferred, modified, or reproduced in any manner without the prior written consent of ACI Worldwide Inc.

ACI, BASE24, BASE24-atm, BASE24-pos, BASE24-telebanking, BASE24-teller, and NET24-XPNET are trademarks or registered trademarks of ACI Worldwide Inc., Transaction Systems Architects, Inc., or their subsidiaries.

Other companies' trademarks, service marks, or registered trademarks and service marks are trademarks, service marks, or registered trademarks and service marks of their respective companies.

Print Record

07/2000, Version 3 (Reissue)

03/97, Version 2 (Reissue)

08/95, Version 1 (New)

Contents

Preface	vii
Conventions Used in this Manual	xiii
Section 1	
Introduction	1-1
Integrated Endpoint Process	1-3
Number of Integrated Endpoint Processes Required	1-4
Single Integrated Endpoint Process for Each XPNET Node	1-4
Multiple Integrated Endpoint Processes for Each XPNET Node	1-5
Partitioned Integrated Endpoint Processes	1-6
Integrated Endpoint Process Usage	1-8
Message-Based Routing Configuration	1-9
Symbolic Destination Routing	1-11
Measure Counters	1-12
Integrated Endpoint Process Configuration Files	1-13
Integrated Endpoint Extended Memory Tables	1-15
Configuration Procedures	1-16
Event Messages	1-18
Event Message Documentation	1-19
Event Suppression Configuration	1-20
Process Identification Number	1-21
Memory Pool Usage	1-22

Section 2

Integrated Endpoint Process Usage	2-1
ITS Environment.	2-2
Procedural BASE24 Environment	2-4
Destination Processes	2-4
IEP Processing	2-4
Configuration Requirements.	2-5
Configuration Example.	2-6
Combination Procedural and ITS Environment	2-9
Overview.	2-9
Configuration Example.	2-9
Pathway Environment.	2-12
Overview.	2-12
Configuration Example.	2-12

Section 3

Message-Based Routing Configuration	3-1
Message-Based Routing Methodology	3-2
The Destination File	3-2
Key Fields.	3-3
Action Fields.	3-3
Selection Hierarchy.	3-4
Setting Up Message-Based Routing	3-6
Message Class Distillation	3-6
Considerations	3-6
Example Assumptions	3-7
Example Destination File Records	3-8
Simple Routing	3-10
Pass-Through Routing	3-13
Broadcast Routing.	3-17
Chained Routing.	3-22
Combined Broadcast and Chained Routing	3-26
Wildcard Routing	3-30
Routing Example 1	3-30
Routing Example 2	3-34

Section 4

Measure Counter Configuration and Reporting	4-1
Measure Counters	4-2
Configuration	4-4
Configuration Steps	4-4
NET24 Performance Monitoring Subsystem	4-6
Starting a Measurement	4-7
Stopping a Measurement	4-8
Viewing Measure Results	4-9
Viewing Procedures	4-9
NET24 Performance Monitoring Subsystem	4-9

Section 5

Integrated Endpoint Process Configuration	5-1
Kernel Configuration	5-2
LCONF Assigns and Params	5-3
Integrated Endpoint LCONF Assigns	5-3
Integrated Endpoint LCONF Params	5-3
Network Environment File	5-4
Network Overload Attributes	5-4
Station Attributes	5-5
DESTINATION Attribute	5-5
SOURCECLASS Attribute	5-6
SERVICE Attribute	5-6
ITS Station Example	5-6
Procedural Station Example	5-7
Process Attributes	5-7
SOURCECLASS Attribute	5-8
SERVICE Attribute	5-8
ITS Process Example	5-8
IEP Example	5-9
Procedural Process Example	5-9
Pathway Configuration File	5-10

Appendix A

Destination File (DEST) A-1

 Destination File Screen Access..... A-2

 Function Keys..... A-4

 DEST Screen..... A-5

 Screen 1..... A-6

Glossary Glossary-1

Index..... Index-1

Preface

This manual provides instructions and guidelines for configuring an Integrated Endpoint process in the integrated transaction services (ITS) environment. The information in this manual enables users to determine, set up, and configure the appropriate message-based and symbolic destination routing required for their use of the Integrated Endpoint process. It includes many examples of the various ways in which the Destination File (DEST) can be configured as well as information about the purpose of the fields on the DEST screen, the codes that can be placed in these fields, and their default values and lengths.

This manual also discusses the general configuration requirements for an Integrated Endpoint process in the following files:

- Integrated transaction services (ITS) kernel configuration files
- Logical Network Configuration File (LCONF)
- Measure Configuration file
- Network Environment File (NEF)

Note: Symbolic destination routing is dictated solely by the logic built into the Endpoint Class Handler object. The DEST is not used for symbolic destination routing.

Audience

The *BASE24 IEP Configuration Manual* is intended as a reference source for users responsible for configuring the message-based and symbolic destination routing functions of an integrated transaction services (ITS) environment as well as the general configuration requirements for an Integrated Endpoint process. The manual serves as a reference guide for determining the message-based and symbolic destination routing needs of an application, implementing an appropriate message-based routing configuration, and configuring the Integrated Endpoint process.

Prerequisites

Readers should be familiar with the *BASE24 ITS Kernel Files Maintenance Manual* and the *BASE24 ITS Kernel Configuration Manual* before reading this manual. The *BASE24 ITS Kernel Files Maintenance Manual* describes each of the screens used to configure processes and objects in the integrated transaction services (ITS) environment. The *BASE24 ITS Kernel Configuration Manual* defines key concepts and terminology for the ITS environment.

Readers of this manual should also be familiar with the Tandem Measure and TACL products.

Additional Documentation

The BASE24 documentation set is arranged so that each manual presents a topic or group of related topics in detail. When one manual in the set presents a topic that has already been covered in detail in another manual, the topic is summarized and the reader is directed to the other manual for additional information. Information has been arranged in this manner to be more efficient for readers who do not need the additional detail and at the same time provide the source for readers who require the additional information.

This manual contains references to the following BASE24 publications:

- The *BASE24 Core Files and Tables Maintenance Manual* describes the screen used to configure Measure user counters for an interface instance.
- The *BASE24 CRT Access Manual* describes the BASE24 Logon screen and the Virtual Menu, which are used to access the Destination File (DEST) screen.
- The *BASE24 IEP Objects Reference Manual* describes each of the integrated endpoint (IEP) objects from which an Integrated Endpoint process is composed and shows how each IEP object is used in transaction processing.
- The *BASE24 ITS Kernel Objects Reference Manual* describes the integrated transaction services kernel objects. These objects are used when building any process in an integrated transaction services (ITS) environment.
- The *BASE24 ITS Kernel Files Maintenance Manual* describes the integrated transaction services configuration and control screens. These screens are used when configuring or controlling any integrated transaction services-based process, such as the Integrated Endpoint process.

- The ***BASE24 ITS Kernel Configuration Manual*** describes the procedures for configuring the integrated transaction services (ITS) environment. These procedures are used when configuring the Integrated Endpoint process.
- The ***BASE24 Logical Network Configuration File Manual*** contains detailed descriptions of LCONF assigns and params used by the Integrated Endpoint process.

This manual also references the following NET24-XPNET and NET24 publications:

- The ***NET24 Performance Monitoring Manual*** provides detailed information on ACI performance monitoring facilities and procedures.
- The ***NET24-XPNET Event Management Manual*** contains detailed information on modifying template source files and the Tandem ViewPoint EVENTCX file.
- The ***NET24-XPNET Network Control Operations Guide*** and the ***NET24-XPNET Network Control Command Reference Manual*** contain detailed information on commands and attributes used to configure processes and stations.

This manual references the following Tandem manuals:

- The ***Measure Reference Manual*** and ***Measure User's Guide*** contain detailed information on the Measure command interface (MEASCOM) and its commands.
- The ***ViewPoint Manual*** contains information on the EVENTCX file.

Software

This manual documents standard processing for Integrated Endpoint Process release 1.3.

Customer-specific code modifications (that is, CSMs) may result in processing that differs from the material presented in this manual. The customer is responsible for identifying and noting these changes.

Manual Summary

The following is a summary of the contents of this manual.

“Conventions Used in this Manual” follows this preface and describes notation and documentation conventions necessary to understand the information in the manual.

Section 1, “Introduction,” introduces message-based and symbolic destination routing, provides an overview of the message-based routing methods, and introduces the message-based routing configuration facilities.

Section 2, “Integrated Endpoint Process Usage,” describes how the Integrated Endpoint process can be integrated into various processing environments.

Section 3, “Message-Based Routing Configuration,” provides examples of each way in which message-based routing can be implemented.

Section 4, “Measure Counter Configuration and Reporting,” describes how to configure Measure user counters on behalf of the Integrated Endpoint process, and how to start a measurement, stop a measurement, and view measurement results.

Section 5, “Integrated Endpoint Process Configuration,” identifies each of the files used to configure the Integrated Endpoint process and indicates what data needs to be configured in each file.

Appendix A, “Destination File (DEST),” describes the Destination File (DEST) maintenance screen and its fields.

A glossary that contains definitions of terms used in this manual follows the last appendix.

Publication Identification

Two entries appearing at the bottom of each page uniquely identify this BASE24 publication. The publication number (for example, OE-AE000-01 for the ***BASE24 IEP Configuration Manual***) appears on every page to assist readers in applying updates to the appropriate manual. The publication date (for example, 07/2000 for July, 2000) identifies the last time material on a particular page has been revised. This date can be used to verify that the reader has the most current information and is particularly helpful when dealing with HELP24 to answer operating questions.

The print record, located on the page immediately following the title page, is also helpful when verifying that a manual contains the most current information. An entry is added to the print record each time a BASE24 publication is modified in any way. Each print record entry includes the date of the change, a unique version number, and the type of change (new, update, reissue, or preliminary). Version 1 is assigned to a *new* manual. Later versions can be updates or reissues, depending on the changes being made. An *update* typically involves changes on a small number of pages throughout the manual. A *reissue* occurs when all pages of the manual are replaced because changes affect a large number of pages. A manual is identified as *preliminary* when it is released to customers on a limited basis, but it has not yet reached final form.

ACI Worldwide Inc.

Conventions Used in this Manual

This section explains how syntax descriptions, screen field descriptions, and unlabeled fields on screens are documented in this manual. It also explains a change in Nucleus process terminology.

Syntax Descriptions

Syntax descriptions in this manual use several symbols and conventions to denote how commands must be entered. These conventions are described in the table below.

Notation	Meaning
UPPERCASE LETTERS	Indicate key words and literals that you must enter exactly as shown.
<i>lowercase italics</i>	Identify variables that you must provide.
Brackets []	Enclose optional syntax items that you can enter in the command.
Braces { }	Enclose a group of items from which you are required to choose one item. The items are separated within the braces by a vertical bar ().
Vertical bar	Separates alternatives in a list of items.
Ellipsis ...	Indicates the immediately preceding syntactical item can occur one or more times.
Punctuation	Must be entered as shown. This includes parentheses, commas, semicolons, and other symbols not previously described.

Spaces	Are required as delimiters wherever it would be otherwise impossible to discriminate between adjacent words or symbols. You can insert additional spaces between any adjacent words or symbols, but not within a word or multiple character symbol.
Indented lines	Indicates a continuation of the preceding line of syntax. If the syntax of a command is too long to fit on a single line, each continuation line is indented.

Field Descriptions

Each field appearing on a file maintenance screen is listed by name and then described. Field descriptions in this manual briefly summarize the contents, purpose, and permissible values, as shown in the following example from the Destination File (DEST).

Example

SOURCE CLASS — The source class associated with the process or station from which a message originates. This field must contain the same value as the SOURCECLASS or CLASS attribute defined for the process or station in its NEF definition. A source class provides a means of grouping similar endpoints as one logical entity for message routing.

The SOURCECLASS attribute is only available for systems running under XPNET 2.1 or greater. The value of the SOURCECLASS attribute can be up to four characters in length, while the value of the CLASS attribute can be up to 16 characters in length. If both the SOURCECLASS and CLASS attributes are defined for a process or station in the NEF, only the value of the SOURCECLASS attribute should be configured in the DEST.

This field, in combination with the MESSAGE CLASS and DESTINATION fields, is the primary key to DEST records.

Example: IAS
Field Length: 1–16 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: DEST.PRIKEY.SRC-CLAS

Explanation

Each field description is completed by one or more of the following items of information:

Item	Description
Example	Illustrates a possible entry for the field to further clarify the value or values that can be entered.
Field Length	Specifies the size of the field and the type of characters that can be entered. This length refers to the field and valid values for the file maintenance screen, not the field in the DDLs. Possible values are alphabetic, alphanumeric, hexadecimal, and numeric. The term <i>alphanumeric</i> includes all alphabetic, numeric, and special characters that can be entered from a keyboard without using a control sequence. The term <i>hexadecimal</i> includes all numbers and the letters A through F. When a field value cannot be modified by the operator, the field length is System-protected.
Occurs	Indicates the number of times the field can be displayed on the screen. This information is provided only when the field can be displayed multiple times.
Required Field	Specifies whether a value has to be entered in the field. Possible values are Yes and No. Some fields are required only under certain conditions. In this case, the entry is Yes, followed by the conditions that determine when the field is required.
Default Value	Specifies the value that is automatically placed in the field when the screen is first displayed or when the F8 key is pressed to clear the screen.
Data Name	Provides the DDL name associated with the field appearing on the screen. Data names are included in the documentation to assist in communicating screen and field issues to your technical staff. Note that screen data is not always stored in the BASE24 database as it appears on the screen or as it is described in the field description. If you need information on how screen data is actually stored, consult the DDLs.

Unlabeled Fields on Screens

Angle brackets (< >) indicate an unlabeled field on a screen. An unlabeled field is a field that is present on a screen but is not preceded by an identifying literal label. For the purposes of documenting the field, a label has been assigned and appears inside the angle brackets. A multiple line unlabeled field is displayed as a shaded area and also has a label in angle brackets.

In the field descriptions for the screen, the unlabeled field appears according to its place on the screen and is identified by the same label.

Unlabeled fields are not included in the index by field name; they appear by subject in the main index.

Nucleus Process Terminology Change

There is one process that is the hub of an XPNET node. It manages message queues; controls all lines, stations, and processes in the system; and serves as an interprocess interface. Throughout this manual, this process is referred to as the XPNET process.

In other manuals, you may see references to the Nucleus process as performing these functions. These two terms (Nucleus process and XPNET process) are synonymous. The Nucleus process and the XPNET process perform the same functions and processing. As manuals are reissued, references to the Nucleus process will be replaced with references to the XPNET process.

Section 1

Introduction

This section introduces you to the Integrated Endpoint process and its configuration requirements. The Integrated Endpoint process serves as an interface between endpoints and applications in the integrated transaction services (ITS) environment. An endpoint is an entity through which messages are sent or received, such as a device, interchange, or host. An endpoint is physically represented by a process or station on the XPNET network and is logically represented by an endpoint class.

The Integrated Endpoint process routes messages received from an endpoint to the appropriate message processing destination(s) based on the message source and the message contents. The process can also send messages to a single symbolic destination determined by the process. The first routing method is known as message-based routing and is dictated by the configuration of the Destination File (DEST). The second routing method, known as symbolic destination routing, is dictated solely by the message contents and the logic included within the Integrated Endpoint process itself.

This section includes discussions of the following topics:

- The number of Integrated Endpoint processes required to meet transaction processing demands
- Overview of message-based routing configuration and routing methods
- Overview of symbolic destination routing
- Introduction to Measure counters and their use in the Integrated Endpoint process
- Introduction to the files used to configure an Integrated Endpoint process
- Introduction to the extended memory tables used by the Integrated Endpoint process
- Overview of the procedures used to configure an Integrated Endpoint process
- Discussion of Integrated Endpoint process event messages and how event messages are documented

- Overview of the event suppression configuration requirements for the Integrated Endpoint process
- Process identification number and memory pool usage for the Integrated Endpoint process

Integrated Endpoint Process

The Integrated Endpoint process can perform the following functions.

- Context management
- Message-based routing
- Symbolic destination routing
- Measure counter manipulation
- Network overload management

Message-based routing configuration is described in section 3 and Measure counter configuration is described in section 4. Symbolic destination routing is described later in this section. The other functions listed above are described in the *BASE24 IEP Objects Reference Manual*.

The Integrated Endpoint process is a satellite process to the XPNET process that functionally, or logically, resides between the XPNET process and an application. The Integrated Endpoint process is an object-oriented process developed under the integrated transaction services object-oriented architecture.

The Integrated Endpoint process determines the destinations of transaction messages received from one or more endpoints. Destinations are determined by the message source (source class) and the message type (message class) configured in the DEST. Symbolic destinations can also be determined by the logic built into the Endpoint Class Handler object. The Integrated Endpoint process routes messages to all destinations determined and optionally updates Measure counters for all messages processed.

The Integrated Endpoint process also maintains transaction context and the timer associated with the transaction context for certain transaction phases. This includes any phase of the transaction for which another message is expected immediately in return to the current message being routed. For example, context is stored when a transaction response message is routed to an acquiring endpoint and an acknowledgment is expected immediately in return. The context is retrieved when the acknowledgment is received.

The Integrated Endpoint process can be used with procedural applications as well as ITS applications.

Note: Code changes to existing procedural BASE24 products may be required to successfully incorporate the Integrated Endpoint process.

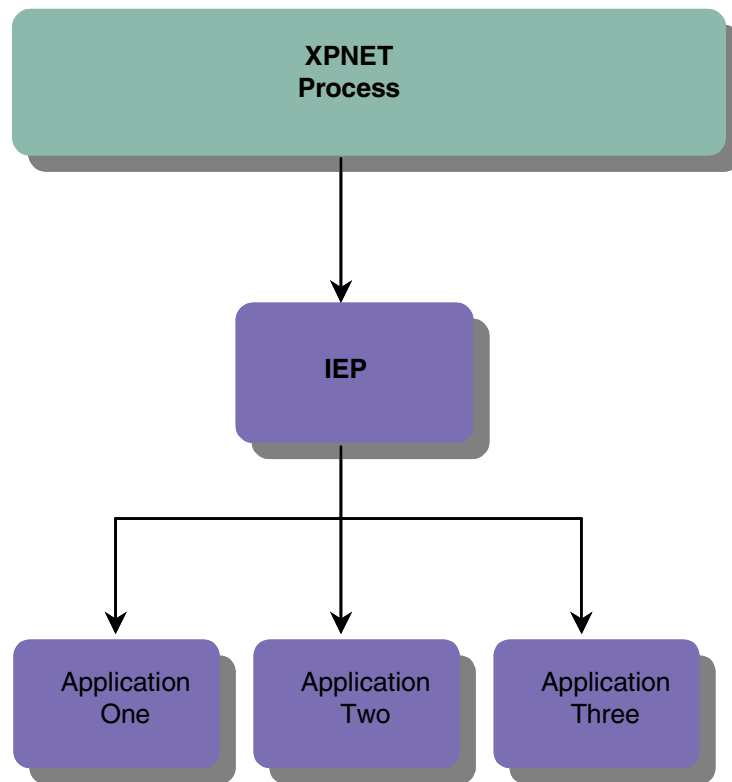
Number of Integrated Endpoint Processes Required

The Integrated Endpoint process can be used with simple configurations such as a single Integrated Endpoint process for each XPNET node to complex configurations using multiple Integrated Endpoint processes provided for each XPNET node. The number of Integrated Endpoint processes used is dependent upon many factors, including your transaction throughput requirements, contingency requirements when one or more Integrated Endpoint processes are unavailable, as well as other business requirements. This subsection discusses some of the issues involved when determining how many Integrated Endpoint processes are required to meet the demands of your application environment.

Single Integrated Endpoint Process for Each XPNET Node

The following is an example of the simplest use of the Integrated Endpoint process with one Integrated Endpoint process configured for each XPNET node. In this configuration, all requests from all of the endpoints in the XPNET node are routed through the same Integrated Endpoint process. For example, if you had interfaces to a host and to an interactive voice response system, the Integrated Endpoint process would contain both a Host Endpoint Class Handler object and a Voice Response Unit Endpoint Class Handler object, respectively.

A single Integrated Endpoint process for each XPNET node configuration is depicted in the illustration on the following page. A key to the abbreviations follows the illustration.

**Key**

IEP = Integrated Endpoint process

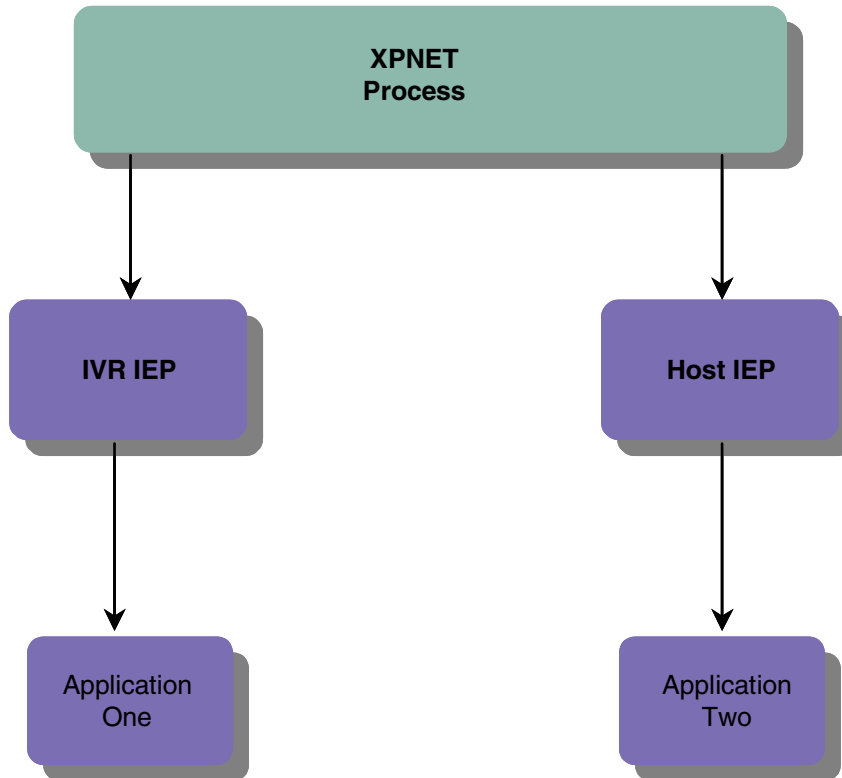
Multiple Integrated Endpoint Processes for Each XPNET Node

A more complex Integrated Endpoint process configuration involves two or more Integrated Endpoint processes for each XPNET node. Each process contains an Endpoint Class Handler object specific to a particular endpoint class. This type of configuration is depicted in the illustration on the following page and embodies the following advantages over a single Integrated Endpoint process for each XPNET node configuration:

- This configuration is easier to manage because the Integrated Endpoint processes can be maintained on different schedules. Software fixes can be applied without bringing down the entire system.

- This configuration is more resilient. If one Integrated Endpoint process fails, service is still available through the other Integrated Endpoint process, although not to a particular endpoint if that endpoint's Integrated Endpoint process is unavailable.
- This configuration can handle a higher volume of traffic than the single Integrated Endpoint process configuration.

A key to the abbreviations follows the illustration



Key

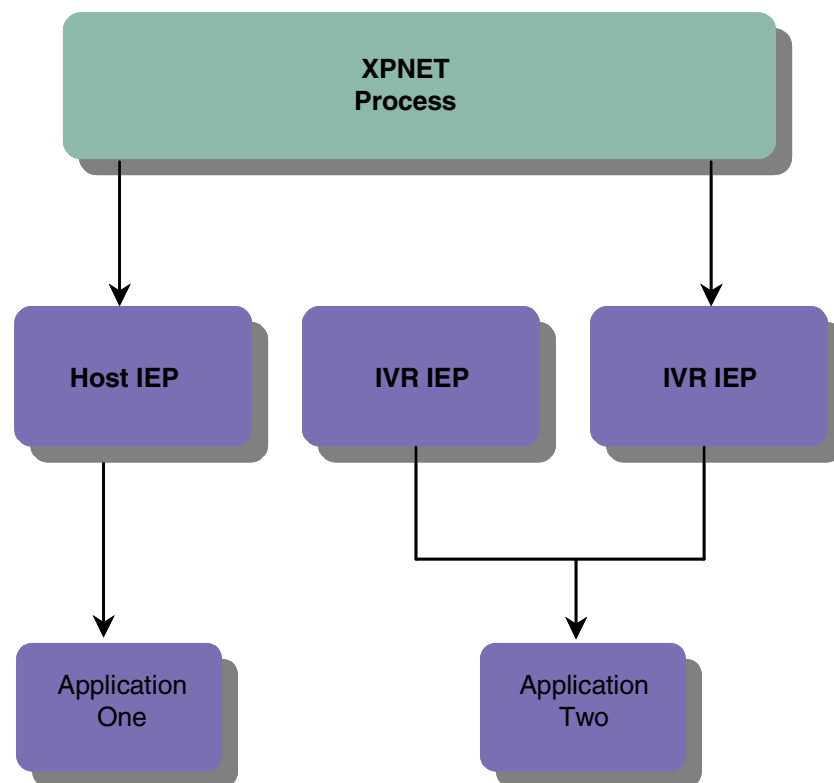
IEP = Integrated Endpoint process
IVR = Interactive voice response system

Partitioned Integrated Endpoint Processes

The Integrated Endpoint process cannot be replicated as a server class on a copy of the XPNET process. The physical endpoints that comprise an endpoint class, however, can be partitioned across multiple Integrated Endpoint processes. For example, a specific interactive voice response system (IVR) station may communicate with one and only one Integrated Endpoint process. However, by

having multiple Integrated Endpoint processes with each process containing a Voice Response Unit Endpoint Class Handler object, the traffic from all IVR stations can be effectively partitioned, or spread, across several different Integrated Endpoint processes. This type of configuration option allows multiple Integrated Endpoint processes to be configured to handle high volume traffic from a single endpoint class (for example, IVR). Partitioning of Endpoint Class Handler objects allows multiple Integrated Endpoint processes to provide parallel processing for the endpoints on an XPNET node, thereby eliminating the possibility of the Integrated Endpoint process slowing down transaction processing.

The illustration below depicts a partitioned Integrated Endpoint process configuration. A key to the abbreviations follows the illustration.



Key

IEP = Integrated Endpoint process
IVR = Interactive voice response system

Integrated Endpoint Process Usage

The Integrated Endpoint process can be used with procedural BASE24 software, ITS software, a combination of procedural and ITS software, and with Pathway-based software. Each of these integration scenarios is described in section 2.

Message-Based Routing Configuration

The message-based routing processing performed by the Integrated Endpoint process is controlled by records in the DEST. This file allows you to use several different routing methods, each of which is described briefly below and in detail in section 3. The DEST screen and its field descriptions are provided in appendix A.

The DEST allows you to use the following routing methods:

- Simple
- Pass-Through
- Broadcast
- Chained

Simple Routing — Routes messages directly to a local authorization entity. With this method, the transaction message only needs to be processed by a single local authorization entity. It does not need to be passed on from one destination to another (chained routing), sent to multiple applications in parallel (broadcast routing), or passed through to an external authorization entity (pass-through routing).

Pass-Through Routing — Routes messages directly from one station to another, without entering an application. The transaction message is simply passed through the system.

Broadcast Routing — Routes a message simultaneously to more than one destination. Thus, broadcast routing allows you to have simultaneous processing of a single transaction by more than one application.

Note: The Integrated Endpoint process does not coordinate responses from messages that are broadcast. This is an application responsibility.

Chained Routing — Routes individual transactions from one application to another in series. In chained routing, a single transaction message chains through multiple applications. Each time the transaction is sent back to the Integrated Endpoint process, the source class or message class is changed, allowing the transaction message to be routed to a different destination.

Combined Broadcast and Chained Routing — Routes messages using a combination of both broadcast and chained routing. This configuration technique provides the most flexibility in transaction routing. Messages can be broadcast to multiple applications while also being chain routed from one application to another.

Symbolic Destination Routing

Symbolic destination routing allows the Integrated Endpoint process to send a message to a single symbolic destination based on the message contents and the logic built directly into the Endpoint Class Handler object for the process. The DEST is not used with this type of routing. The destination is determined solely from the message contents by the process itself. The object can also determine the message class, if the message is also routed to additional destinations using message-based routing. Once the Endpoint Class Handler object has determined the symbolic destination and message class for the message, it sends the Message Route Symbolic member function to the Endpoint Class Handler Services object. Only one symbolic destination can be specified in this member function. The Endpoint Class Handler Services object then performs message-based routing if required. The object creates an external message for each message determined during message-based routing as well as for the single specified symbolic destination.

Symbolic destination routing can be used with or without message-based routing. Every message processed by the Integrated Endpoint process must have at least one destination.

Any desired logic can be included in the Endpoint Class Handler object to determine the symbolic destination for a message. For example, the Endpoint Class Handler object could be constructed to take advantage of destination string routing currently used with many procedural BASE24 products.

Symbolic destination routing can be useful when you are incorporating the Integrated Endpoint process into an existing procedural application environment where station-to-station routing (pass-through routing) is required. After an incoming message from a station has been processed by the appropriate procedural BASE24 Device Handler, Host Interface, or Interchange Interface process, it can be sent to the Integrated Endpoint process for both symbolic destination and message-based routing. Symbolic destination routing can be used to send the message to a single symbolic station destination, while message-based routing can be used to send the message to multiple procedural or ITS destinations.

The Integrated Endpoint process typically follows the procedural Device Handler or interface process in the transaction path so that device- or interface-specific data is maintained.

Measure Counters

The Integrated Endpoint process uses the Tandem Measure product to optionally track Integrated Endpoint processing statistics. The Endpoint Class Handler and Endpoint Class Handler Services objects maintain the processing statistics.

The Measure counters used by the Integrated Endpoint process as well as instructions for configuring and reporting them are provided in section 4.

Integrated Endpoint Process Configuration Files

Several different files must be configured to implement an Integrated Endpoint process. This subsection introduces you to each of these files and tells you where to find more information on configuration of the file.

Destination File (DEST) — Defines the message-based routing destinations for messages based on the source class and message class associated with a given message. The DEST is described in appendix A. Examples for configuring the file for various routing scenarios are provided in section 3.

Kernel Object Configuration Files — All objects used by the Integrated Endpoint process must be configured in the ITS kernel object configuration files. The kernel file configuration records needed on behalf of the Integrated Endpoint process are described in section 5. Each of the kernel configuration files is described in the *BASE24 ITS Kernel Files Maintenance Manual*. Instructions for configuring these files are provided in the *BASE24 ITS Kernel Configuration Manual*.

For details on the configuration requirements of kernel objects used by the Integrated Endpoint process, refer to the *BASE24 ITS Kernel Objects Reference Manual*.

For details on the configuration requirements of Integrated Endpoint process objects, refer to the *BASE24 IEP Objects Reference Manual*.

Logical Network Configuration File (LCONF) — Contains the assigns and params used by Integrated Endpoint process objects. Each of these assigns and params is listed in section 5. Refer to the *BASE24 Logical Network Configuration File Manual* for detailed descriptions of LCONF assigns and params.

Measure Configuration File — Configures Measure user counters used by the Integrated Endpoint process. Instructions for configuring Measure user counters are provided in section 4.

Network Environment File (NEF) — Contains the configuration for each Integrated Endpoint process running under control of the XPNET process as well as all processes and stations running under the XPNET process that route messages to an Integrated Endpoint process. The source class for a process or station is determined by attributes configured for the process or station using a

network control facility. Instructions for configuring the Integrated Endpoint process and processes or stations that route messages to an Integrated Endpoint process are provided in section 5.

Core Configuration Files — Depending upon the application, the configuration of some information in core files may be required on behalf of the Integrated Endpoint process. For example, the BASE24-telebanking application requires the configuration of Measure user counter names used by the Integrated Endpoint process on the VRU Configuration Data (VCD) screen if integrated endpoint processing statistics are collected. This data is stored in the ITS Interface Configuration File (IFCF) and is accessed from the Interface extended memory table. For more information on the Integrated Endpoint process configuration requirements for the IFCF, refer to section 4.

Procedural Configuration Files — If the Integrated Endpoint process is used with procedural BASE24 applications, certain configuration files may need to be configured to route to the Integrated Endpoint process. For example, if messages received from a host are to be processed by the Integrated Endpoint process, the Host Configuration File (HCF) must be configured to route incoming messages to the Integrated Endpoint process. The exact configuration files impacted are application-specific and are beyond the scope of this manual.

Integrated Endpoint Extended Memory Tables

Several extended memory tables are used for integrated endpoint processing. Each of these tables is described below.

Destination File (DEST) extended memory table — Contains information from the Destination File (DEST). The DEST is used to define the destinations for messages based on the source class and message class associated with the given message. One record must exist in this file for each source class, message class, and endpoint destination combination used in your system.

Anytime a change is made to the DEST, the DEST extended memory table must be rebuilt using the Extended Memory Table Build utility. After the DEST extended memory table is rebuilt, it is not used in processing until it is reallocated by the objects that use it. This is accomplished by sending the EMT WARMBOOT or LCONF WARMBOOT command to the Destination object within the Integrated Endpoint process.

The DEST file maintenance screen and its fields are described in appendix A.

Instructions for using the EMT WARMBOOT and LCONF WARMBOOT commands are provided in the *BASE24 ITS Kernel File Maintenance Manual*.

Context extended memory table — Stores context for messages that require it. Context is defined as a copy of a message and a timer associated with the message. This table is used by the Context Manager object and is built when the Context Manager object is initialized. It is not built from a source file or table.

Interface extended memory table — Specifies the Measure counter name and the Message Class wildcard options for an interface instance. This information is retrieved by the Endpoint Class Handler object for BASE24-telebanking. This table contains information from the Address Table File (ADRT), External Message File (EMF), Interface Station Table File (IST), and ITS Interface Configuration File (IFCF). The Extended Memory Table Build utility builds this table.

Instructions for running the Extended Memory Table Build utility are provided in the *BASE24 ITS Kernel Configuration Manual*.

Configuration Procedures

This subsection provides a high-level overview of the procedures required to configure an Integrated Endpoint process. Refer to the appropriate section of this manual for detailed information on each procedure.

Note: These procedures assume the previous installation of all relevant ITS, integrated endpoint, and core software.

1. Define the message-based and symbolic destination routing requirements for your implementation of the Integrated Endpoint process. For message-based routing, you must define the source class, message class, and destination for each transaction to be processed. Your intended use of the Integrated Endpoint process should fall under one of the message-based routing scenarios provided in section 3 or a symbolic destination routing scenario discussed under the “Symbolic Destination Routing” heading elsewhere in this section.
2. Configure any LCONF assigns or params used by the kernel, integrated endpoint, and core objects that comprise the Integrated Endpoint process. Instructions for configuring the LCONF are provided in section 5.

Refer to the *BASE24 Logical Network Configuration File Manual* for detailed assign and param descriptions.

3. Configure each Integrated Endpoint process to the XPNET system as well as for each process and station running under control of the XPNET process that routes to the Integrated Endpoint process. Detailed instructions are provided in section 5.
4. Configure the ITS kernel environment for each Integrated Endpoint process as well as for each Integrated Server process that either routes to the Integrated Endpoint process or receives messages from the Integrated Endpoint process.

The specific kernel configuration requirements for the Integrated Endpoint process are provided in section 5.

For more information on the kernel environment configuration files and screens, refer to the *BASE24 ITS Kernel Files Maintenance Manual*.

5. Configure all of the message-based routing information defined in step 1 in the DEST.

For detailed information on the DEST screen and its fields, refer to appendix A.

6. If the Integrated Endpoint process is to be incorporated into an existing procedural BASE24 environment, make any procedural configuration file changes required to route messages to the Integrated Endpoint process. The exact changes needed are application-specific and beyond the scope of this manual.

Note: Code changes to existing procedural BASE24 products may also be required to successfully incorporate the Integrated Endpoint process.

Event Messages

ACI supplies template source files which provide information about each Event Management Service (EMS) event generated by Integrated Endpoint process objects. The following Integrated Endpoint object template source files are provided with the Integrated Endpoint process. Integrated Endpoint object template source files residing on OC13 subvolumes are for application-specific Endpoint Class Handler objects.

- The CTXMPS file on the OE13CTXM subvolume provides information about Context Manager object events.
- The DESTPS file on the OE13DEST subvolume provides information about Destination object events.
- The EPCSPS file on the OE13EPCS subvolume provides information about Endpoint Class Handler Services object events.
- The GENEPS file on the OE13GENE subvolume provides information about Generic Endpoint Class Handler object events.
- The IMTEPS file on the OC13IMT subvolume provides information about Internal Message Translate Endpoint Class Handler object events.
- The MISOPS file on the OE13MISO subvolume provides information about Message ISO object events.
- The VRUEPS file on the OC13VRU subvolume provides information about Voice Response Unit Endpoint Class Handler object events. This object handles messages received from a remote banking interface endpoint device such as an interactive voice response system (IVR).

If desired, you can modify these files to change the text of event messages generated by the Integrated Endpoint process. The ***NET24-XPNET Event Management Manual*** contains detailed information on modifying template source files and the Tandem ViewPoint EVENTCX file. The EVENTCX file is an optional file which contains event detail information about event messages generated by an application subsystem. For more information on the EVENTCX file, see the Tandem ***ViewPoint Manual***.

Object template source files for kernel objects used by Integrated Endpoint processes are discussed in the ***BASE24 ITS Kernel Configuration Manual***.

Event Message Documentation

Event message documentation is available for each of the event messages provided by ACI for Integrated Endpoint process objects. All event messages provided for Integrated Endpoint process objects are contained on the OK13LOGM subvolume. All Integrated Endpoint object event message documentation files in this subvolume are identified with a product module ID, or product group, of ISIE or ISCO as shown in the following table. The ISCO product module ID is used for application-specific Endpoint Class Handler objects.

File Name	Contents
ISCOVRUE	Voice Response Unit Endpoint Class Handler object events Note: This object handles messages received from a remote banking interface endpoint device such as an interactive voice response system (IVR).
ISIECTXM	Context Manager object events
ISIEEPCS	Endpoint Class Handler Services object events
ISIEDEST	Destination object events
ISIEGENE	Generic Endpoint Class Handler object events
ISIEIMTE	Internal Message Translate Endpoint Class Handler object events
ISIEMISO	Message ISO object events

For more information on kernel event message documentation, refer to the *BASE24 ITS Kernel Configuration Manual*.

For more information on core event message documentation, refer to the application-specific transaction processing manual.

See the README file on the OK13LOGM subvolume for instructions on printing event message documentation.

Event Suppression Configuration

This subsection describes how various fields in the Event Suppression File (ESF) must be set to support different types of event suppression for Integrated Endpoint process objects. For detailed information on event suppression configuration procedures, refer to the *BASE24 ITS Kernel Configuration Manual*. For detailed information on the Event Suppression File (ESF) and its fields, refer to the *BASE24 ITS Kernel Files Maintenance Manual*.

The following fields in the ESF must be set properly to support all levels of event suppression:

SSID — Must be set to the value assigned to Integrated Endpoint objects in the EMCLDDLS file in the \$SYSTEM.SCRIBE subvolume. The current SSID values for Integrated Endpoint process objects are described in the following table. For kernel-specific SSIDs, refer to the *BASE24 ITS Kernel Configuration Manual*. For core-specific SSIDs, refer to the application-specific transaction processing manual.

SSID	Description
739	Voice Response Unit Endpoint Class Handler object
752	Internal Message Translate Endpoint Class Handler object
756	Context Manager object
757	Destination object
758	Endpoint Class Handler Services object
760	External Message ISO object
783	Generic Endpoint Class Handler object

Event Number — Must be set to the value specified by the application for the event. These are provided in the online event message documentation files on the OK13LOGM subvolume or in the template files provided on the OC13 subvolumes for each object.

Process Identification Number

XPNET release 2.1 or greater supports the extended features of the Tandem D-series operating system that enable a process to run with a high process identification number (PIN). A PIN is a unique, system-assigned identifier with a numeric value that identifies processes running in a CPU. The Tandem D-series operating system allows up to 65,534 processes to run simultaneously in a single CPU. High PIN is defined as a process identification number in the range of 256 to 65,534. Low PIN refers to a process identification number in the range of 0 to 254. The XPNET process and all Integrated Server processes that run under control of the XPNET process, including all Integrated Endpoint processes, can be started as either low or high PIN processes.

All processes started by ITS processes (for example, a report process started by the End-of-Period process) are started as low PIN processes only.

The Extended Memory Table Build utility, is started as a low PIN process when executed from the Configuration Control screen (CCS). However, this utility can be started as a high or low PIN process when executed from a TACL prompt.

A process runs with a high PIN when the executable object file contains the HIGHPIN file attribute, or if you start the process from TACL with the HIGHPIN ON attribute.

Note: High PIN functionality for the Integrated Endpoint process is not available if your ITS environment runs under XPNET release 2.0.

Memory Pool Usage

XPNET release 2.1 or greater supports the extended features of the Tandem D-series operating system that enable a process to use a large-memory model as well as a small-memory model. ITS processes running under control of the XPNET process can use either memory model. Using the small-memory model, ITS objects can allocate space in the upper 64K bytes (32K words) of the data stack and use message buffers of 4096 bytes. Using the large-memory model, ITS objects can allocate space within a flat segment. Flat segments allow memory space to be larger than 65,536 bytes, use message buffers of 32,000 bytes, and require a D30 or later operating system running on a Tandem NonStop Series/RISC machine.

A large-memory model toggle set when the Application Network Services object is compiled determines whether the small-memory model or large-memory model is used. If the toggle is set, the large-memory model is used. If the toggle is not set, the small-memory model is used.

Note: The large-memory model is not available if your ITS environment runs under XPNET release 2.0 or if your Tandem system is not RISC-based.

Section 2

Integrated Endpoint Process Usage

This section provides examples for the various ways in which the Integrated Endpoint process can be incorporated into various processing environments. The Integrated Endpoint process is flexible in that it can be incorporated into or used with the following:

- Integrated transaction services (ITS) environments
- A combination of procedural and ITS environments
- Existing procedural BASE24 environments
- Pathway environments

Note: Of the above processing environments, the Integrated Endpoint process is currently used in ITS environments only. The use of the Integrated Endpoint process shown in the other processing environments is strictly hypothetical. The additional processing environment scenarios are provided here to show you how the Integrated Endpoint process can be extended into other processing environments. Additional Endpoint Class Handler objects, code changes, and configuration changes may be required to implement the Integrated Endpoint process in these environments. The example transaction flows provided in this section are for illustrative purposes only. They should not be construed as providing the definitive configuration for a specific application of the Integrated Endpoint process.

The following pages discuss how the Integrated Endpoint process can be incorporated into or used with each of these environments and what needs to be configured on its behalf for each environment. Each of these scenarios assumes the use of an endpoint-specific Endpoint Class Handler object that may or may not already exist, depending upon the endpoint supported.

Although symbolic destination routing is not depicted in this section, it could also be used with any of the above environments.

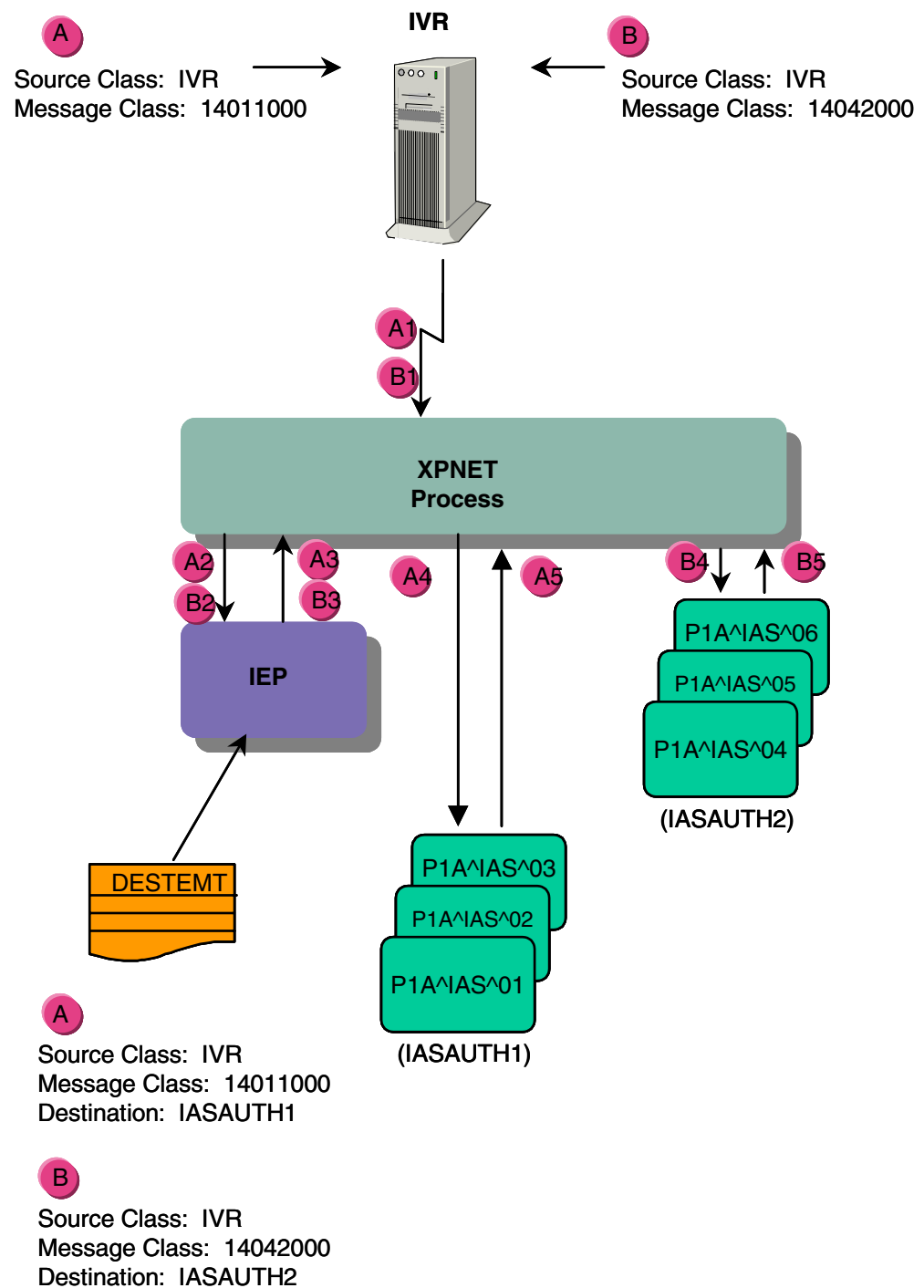
ITS Environment

Use of the Integrated Endpoint process in an ITS environment presents the most straightforward use of the Integrated Endpoint process. In this type of environment, the Integrated Endpoint process is integrated with ITS-based processes only running under control of the XPNET process. Typically, the Integrated Endpoint process will be configured to route messages to services of ITS processes for processing, although the Integrated Endpoint process could just as easily be used to route directly to another XPNET station as discussed later in this section and in section 3 under the “Pass-Through Routing” heading. In the example configuration illustrated on the following page, all messages of class 14011000 received from an interactive voice response system (IVR) are routed to the IASAUTH1 service (A₁–A₅). The first available ITS process in the IASAUTH1 service would then acquire the message and process the transaction. All messages of class 14042000 received from an interactive voice response system (IVR) are routed to the IASAUTH2 service (B₁–B₅). The first available ITS process in the IASAUTH2 service would then acquire the message and process the transaction. All message routing is completely controlled by the configuration of the DEST.

Broadcast, chained, and pass-through routing configurations can also be used for Integrated Endpoint processes running in an ITS environment as discussed elsewhere in this section and in section 3.

For applications comprised of Integrated Server processes, the Integrated Endpoint process is typically placed in the transaction path prior to any message processing by the application.

A key to the abbreviations follows the illustration.



Key

DESTENT = Destination File (DEST) extended memory table
 IEP = Integrated Endpoint process
 IVR = Interactive voice response system

Procedural BASE24 Environment

This subsection describes how the Integrated Endpoint process can be integrated with procedural BASE24 applications (for example, BASE24-atm, BASE24-pos, and BASE24-teller). The Integrated Endpoint process adds message-based routing flexibility not currently available in these applications. Currently, when a transaction is acquired on a station by an XPNET process, it can be routed by the XPNET process based on the source of the message to a single destination or to a service.

Note: The following paragraphs describe the hypothetical use of the Integrated Endpoint process with procedural BASE24 applications. The Integrated Endpoint process provides message-based routing capabilities that are not currently available in procedural BASE24 products. An Endpoint Class Handler object for procedural BASE24 products has not yet been developed. In addition, code changes to existing procedural BASE24 products may be required to successfully incorporate the Integrated Endpoint process.

Destination Processes

With message-based routing provided by the Integrated Endpoint process, more than one destination process (for example, Authorization process) for a transaction can be specified for an endpoint. Thus, individual transactions from a single endpoint can be routed to any one of a number of applications based on the source class and message class of the transaction. This allows smart terminal devices, which can process transactions for multiple applications, to be integrated with existing procedural BASE24 applications. In procedural BASE24 applications without an Integrated Endpoint process, transaction messages from a particular endpoint (for example, Device Handler, Host Interface, or Interchange Interface process) can be sent to one and only one destination.

Message-based routing also allows certain types of messages to be routed to a host or device interface, rather than to an online authorization process.

IEP Processing

When an Integrated Endpoint process receives a message from a procedural BASE24 Device Handler, Host Interface, or Interchange Interface process in internal message format, it does not perform context management processing, since this would duplicate the context management functions already being performed by the source process.

The Integrated Endpoint process performs message-based routing as described in the example and also optionally performs Measure counter manipulation if it is configured to do so.

Configuration Requirements

To implement the Integrated Endpoint process in some procedural BASE24 environments, the terminal data file record for a Device Handler process, the Host Configuration File (HCF) for a Host Interface process, or the Interchange Configuration File (ICF) for an Interchange Interface process typically will need to be modified to make the Integrated Endpoint process the destination for transactions originating from the endpoint. The Destination File (DEST) records for the Integrated Endpoint process can then be configured to route individual transaction messages to multiple applications based on the source class and message class of each transaction message.

In the transaction path, the Integrated Endpoint process typically must follow the procedural Device Handler or interface process. This is required because the Device Handler or interface process typically maintains any data kept for the device or interface such as terminal data file context (for example, message authentication code keys and totals for Device Handler processes). Thus, the Device Handler or interface process must process every message to keep such data synchronized with the device or interface it services. In addition, procedural BASE24 Device Handler processes, Host Interface processes, and Interchange Interface processes typically must process each message to handle message sequencing and message authentication. Messages can be in any format (for example, device-native or internal) when they reach the Integrated Endpoint process, as long as the Integrated Endpoint process contains an Endpoint Class Handler object that can interpret the appropriate message format to build the message class.

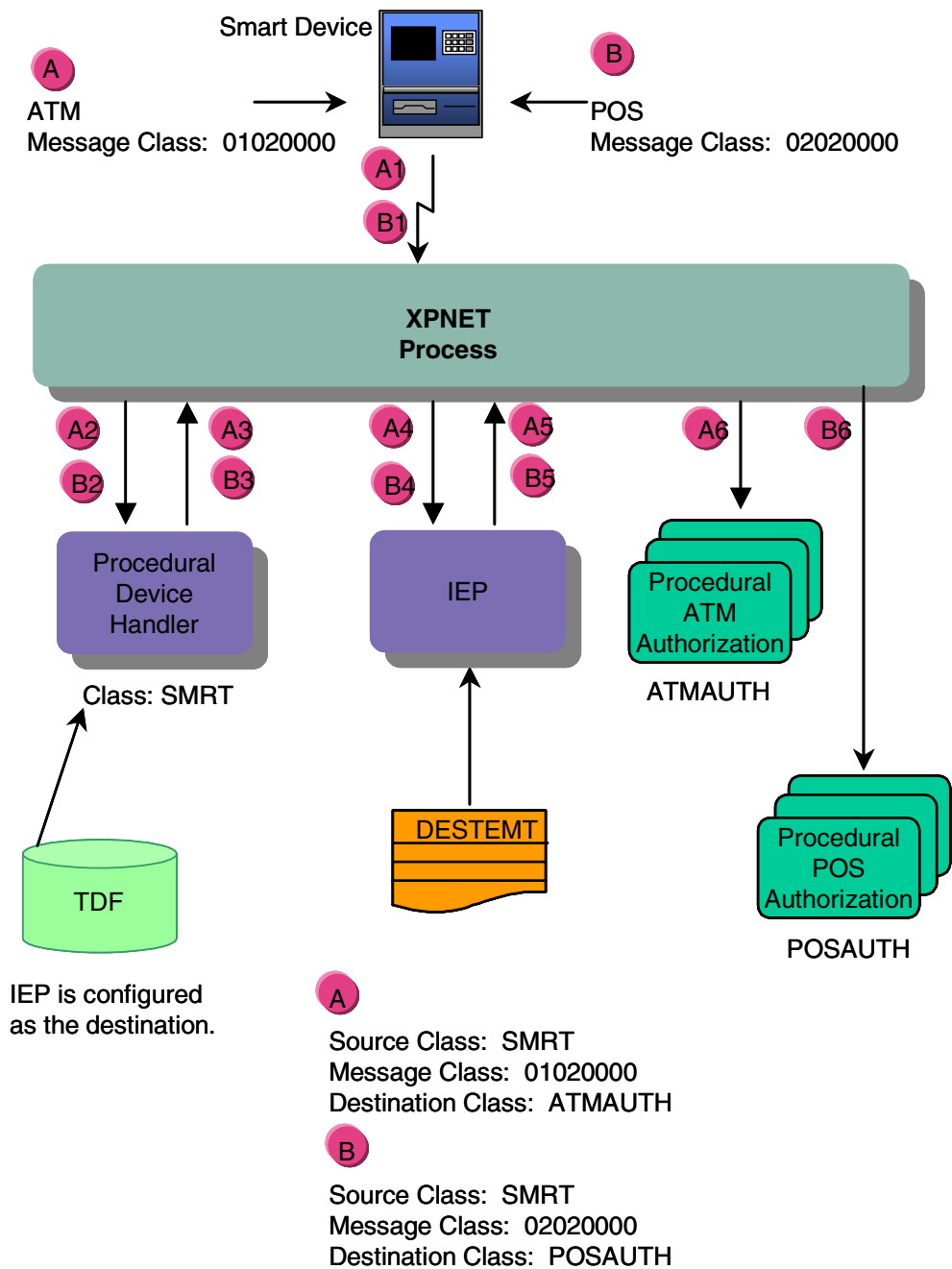
The configuration of the Integrated Endpoint process in a procedural environment is depicted in the configuration example starting on the following page. This example depicts the simple routing of messages from the endpoint directly to online authorization application processes.

Broadcast, chained, and pass through routing configurations can also be used for Integrated Endpoint processes running in a procedural BASE24 environment as discussed elsewhere in this section and in section 3.

Configuration Example

The illustration on the following page shows an example of an Integrated Endpoint process integrated into a procedural BASE24 environment. In this example, the Integrated Endpoint process is used to route transactions from a smart device that can handle both automated teller machine (ATM) and point-of-sale device (POS) transactions. Transaction messages are first processed by a procedural BASE24 Device Handler process before being routed by the Integrated Endpoint process. In this example, messages are converted to internal format by the Device Handler process before they are processed by the Integrated Endpoint process.

A key to the abbreviations follows the illustration. The processing depicted in the illustration is described on the page following the illustration.



Key

ATM	= Automated Teller Machine
DESTENT	= Destination File (DEST) extended memory table
IEP	= Integrated Endpoint process
POS	= Point-of-Sale Device
TDF	= Terminal Data File

As depicted in the preceding illustration, both an ATM transaction (A) and a POS transaction (B) are initiated at a smart terminal device. The Integrated Endpoint process is used to route transactions to the appropriate service of Authorization server processes based on the source class and message class of the transaction message. For this example, assume that the message class is comprised of the product ID, message type, and message variant associated with the message. Transactions are first routed to a Device Handler process (A₁–A₃, B₁–B₃) for message conversion to an internal format, message sequencing, and message authentication. The Integrated Endpoint process is then configured as the destination for the terminal device in the BASE24-atm Terminal Data File (TDF).

Both the ATM and POS transactions are routed to the Integrated Endpoint process for message-based routing (A₄–A₅, B₄–B₅). The Integrated Endpoint process determines that messages with a source class of SMRT and a message class of 01020000 are to be sent to a BASE24-atm Authorization service (ATMAUTH). Messages with a source class of SMRT and a message class of 02020000 are to be sent to a BASE24-pos Authorization service (POSAUTH). The Integrated Endpoint process then delivers the ATM message to the ATMAUTH service (A₆), and the POS message to the POSAUTH service (B₆). The first available process in each service would then acquire the message and process the transaction.

Combination Procedural and ITS Environment

This subsection discusses how the Integrated Endpoint process can be used in an environment where both procedural and ITS applications are used. This scenario shows you how the Integrated Endpoint process can be extended from the ITS environment into existing procedural BASE24 environments.

Note: The following paragraphs describe the hypothetical use of the Integrated Endpoint process with both procedural and ITS BASE24 applications. An Endpoint Class Handler object for procedural BASE24 products has not yet been developed. In addition, code changes to existing procedural BASE24 products may be required to successfully incorporate the Integrated Endpoint process.

Overview

The Integrated Endpoint process can be integrated with both procedural and ITS applications at the same time. This capability allows you to move parts of your current procedural processing environment into the ITS environment as slowly or as quickly as desired. Through the use of an Integrated Endpoint process you can handle message routing from both procedural- and ITS-based endpoints simultaneously. Each Integrated Endpoint process is constructed to handle the specific message types required, whether they be procedural- or ITS-based.

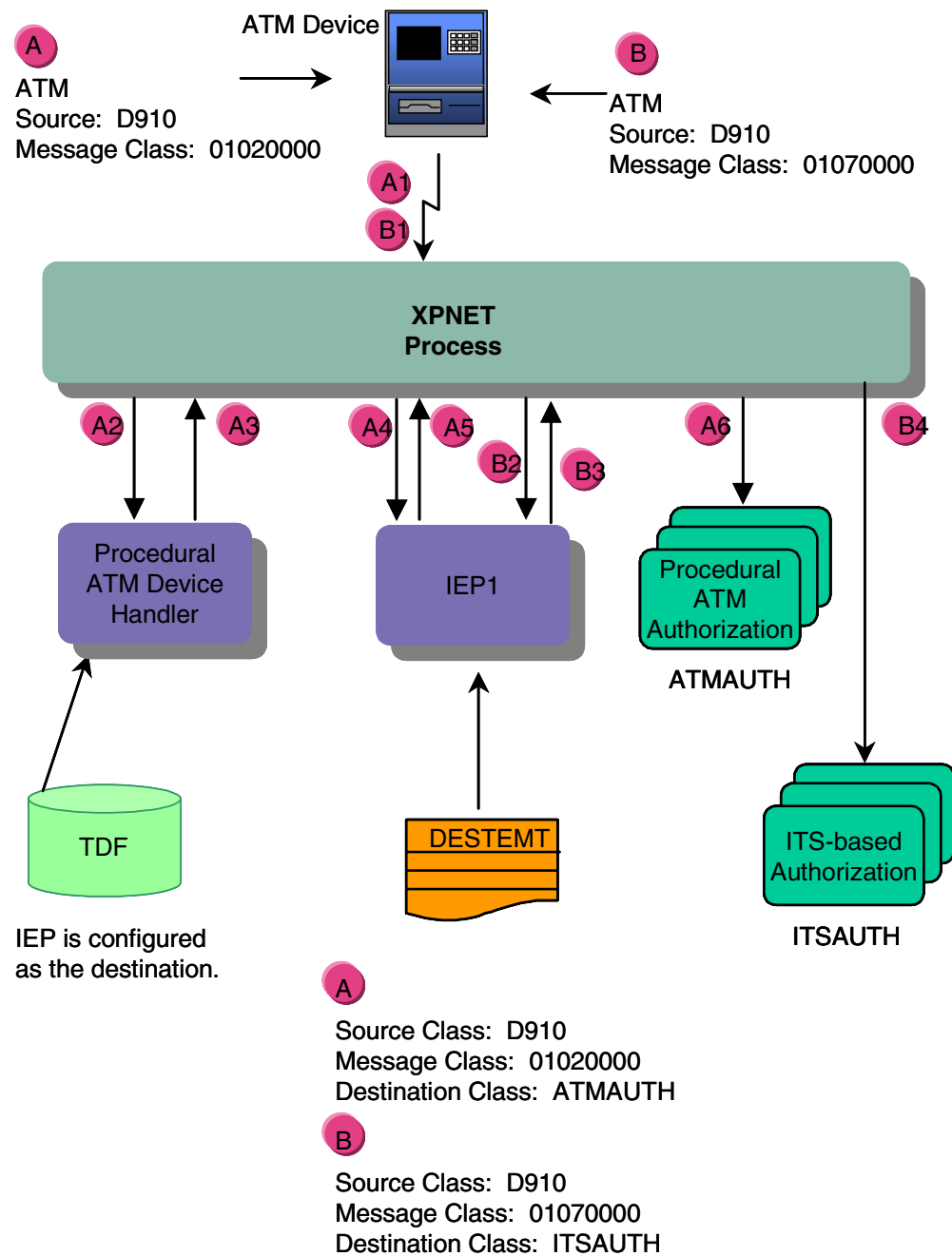
Configuration Example

The following is an example of the Integrated Endpoint process incorporated into a combination procedural and ITS environment. In this configuration example, ATM transaction messages received from an ATM endpoint (for example, a Diebold 910 device) are currently routed to a procedural destination. A new transaction type is being added to the ATM transaction set that will always be routed to a new ITS-based application. Thus, the same Integrated Endpoint process will handle the message routing requirements for both procedural-bound and ITS-bound messages at the same time.

As depicted in the illustration on the following page, both a currently supported ATM transaction (A_1) and a new ITS-supported transaction (B_1) are initiated at an ATM. The currently supported transaction is routed to a procedural authorization process and therefore must be first processed by a procedural ATM Device Handler process (A_2 – A_3). Next, the transaction is routed to an Integrated Endpoint process for message-based routing to a procedural ATM Authorization service (A_4 – A_5). Finally, the transaction is delivered to a service of procedural ATM processes (A_6).

When the new ITS-supported ATM transaction is received, the transaction is immediately sent to the Integrated Endpoint process for message-based routing (B₂–B₃). No front-end processing by an existing procedural Device Handler process is required. The new transaction is then routed directly to a service of ITS-based server processes (B₄) for processing.

A key to the abbreviations follows the illustration.



Key

ATM	= Automated Teller Machine
DESTEMT	= Destination File (DEST) extended memory table
IEP	= Integrated Endpoint process
ITS	= Integrated Transaction Services
TDF	= Terminal Data File

Pathway Environment

This subsection discusses how the Integrated Endpoint process can be incorporated into a system using a Pathway-based application endpoint.

Note: The following paragraphs describe the hypothetical use of the Integrated Endpoint process with a Pathway-based application endpoint. An Endpoint Class Handler object for a Pathway-based application endpoint has not yet been developed. The BASE24-telebanking product does not use the Integrated Endpoint process for transactions acquired from the Pathway-based customer service representative (CSR) terminals.

Overview

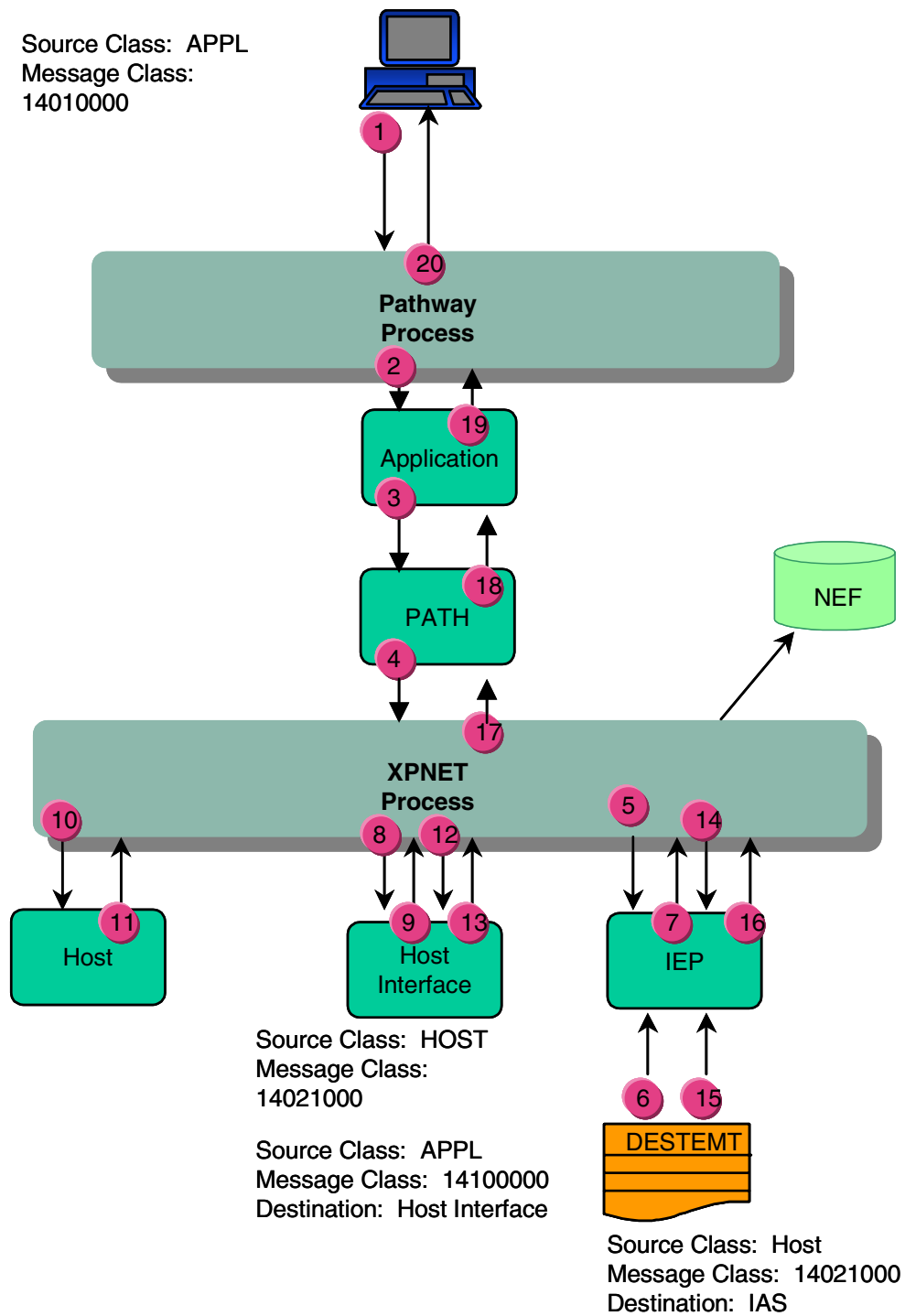
The Integrated Endpoint process can be incorporated with an application running under the Pathway process for messages that are destined for an external authorization entity. In this type of configuration, the application runs under the Pathway process and the Integrated Endpoint process runs under the XPNET process. Currently, the Integrated Endpoint process must always run under the XPNET process. The Integrated Endpoint process is used to route messages to external endpoints for authorization processing that cannot be performed by the Pathway-based application.

The Path Server process is used to allow processes running under the Pathway process to communicate with processes running under the XPNET process.

Configuration Example

The illustration on the following page shows an example transaction flow for an Integrated Endpoint process integrated with a Pathway-based application environment. The steps provided in this flow are provided on the pages following the illustration.

A key to the abbreviations follows the illustration. Processing step descriptions are also provided following the illustration.



Key

PATH	= Path Server process
DESTEMT	= Destination File (DEST) extended memory table
IEP	= Integrated Endpoint process
NEF	= Network Environment File

Processing is as follows:

1. A transaction is initiated at a Pathway terminal.
2. The Pathway process delivers the transaction to an application process running as a Pathway server. The application process determines that the transaction must be authorized on a host.
3. The application process sends the transaction message to the Path Server process for delivery to the XPNET process.
4. The Path Server process delivers the message to the XPNET process using an XPNET station.
5. The XPNET process delivers the transaction to the Integrated Endpoint process based on the destination specified in the NEF for the acquiring station.
6. The source class of the message identifies the appropriate endpoint class handler object (for example, Internal Message Translate Endpoint Class Handler object) as the message handler for this message. The Internal Message Translate Endpoint Class Handler object recognizes the message, determines the message destination from the Destination File (DEST) extended memory table and sends the message to the Host Interface process.
7. The Integrated Endpoint process sends the transaction to the XPNET process for delivery to the Host Interface process.
8. The XPNET process delivers the transaction to the Host Interface process.
9. The Host Interface process converts the message from internal to native format and sends the message to the XPNET process for delivery to the host.
10. The XPNET process delivers the message to the host.
11. The host authorizes the transaction and sends the transaction response to the XPNET process.
12. The XPNET process delivers the transaction response to the Host Interface process.

13. The Host Interface process builds an internally formatted message and sends the response message to the XPNET process for delivery to the Integrated Endpoint Process that originated the request.
14. The XPNET process delivers the transaction response to the Integrated Endpoint process.
15. The Integrated Endpoint process examines the source class and message class of the message and determines that the message should be routed to an application process running under control of the Pathway process.
16. The Integrated Endpoint process sends the transaction response to the XPNET process for delivery to the acquiring station.
17. The XPNET process delivers the transaction response to the acquiring station, which forwards it to the Path Server process.
18. The Path Server process sends the response message to the Pathway-based application process.
19. The application process logs the transaction response and sends it through the Pathway process for delivery to the terminal screen.
20. The Pathway process delivers the transaction response back to the originating terminal where it is displayed on a terminal screen.

ACI Worldwide Inc.

Section 3

Message-Based Routing Configuration

Message-based routing can be configured to meet the demands of virtually any processing environment. Whatever your purpose is for using the Integrated Endpoint process, the configuration of your message-based routing will fall under one or more of the following scenarios depicted in this section.

- Simple routing of transactions to an online application.
- Pass-through routing of transactions from one station to another (station-to-station routing). In this type of configuration, transactions are not processed by an application on the Tandem system.
- Broadcast routing of transactions to two or more applications simultaneously (that is, in parallel).
- Chained routing of transactions to two or more applications sequentially (that is, in series).
- A combination of broadcast and chained routing.
- Wildcard routing. For any of the above scenarios, some or all of the message class subfields can use wildcard characters for routing purposes.

All message-based routing configuration is defined in the Destination File (DEST). This section provides the DEST records required for each of the example scenario configurations. Detailed DEST screen and field descriptions are provided in appendix A.

Note: The example transaction flows provided in this section are for illustrative purposes only. They should not be construed as providing the definitive configuration for any specific application of the Integrated Endpoint process.

Message-Based Routing Methodology

The Destination File extended memory table is built by the Extended Memory Table Build utility from information in the DEST. The Destination File extended memory table contains the message-based routing logic for the Integrated Endpoint process. The Destination object in the Integrated Endpoint process searches the Destination File extended memory table to obtain destinations for incoming transactions received from an endpoint.

The Destination File

The DEST contains the message-based routing logic for the Integrated Endpoint process. You add records to the DEST as needed to support the routing requirements for your application. Each record in the DEST contains the following information:

- A source class
- A message class
- A destination
- A destination object name, if one exists
- A flag indicating whether the destination should expect to receive messages encapsulated within a Transport Layer Object to Transport Layer Object member function request.
- A flag indicating whether the destination should expect to receive messages encapsulated within an Endpoint Class Handler Services to Object member function request.

DEST records can be divided into two parts: key fields and action fields. The key fields allow Integrated Endpoint processes to select the appropriate records from the Destination File extended memory table. The action fields prescribe the routing actions to take once DEST records have been selected.

A brief description for each of the fields in a DEST record is included on the next page. These descriptions summarize the fields for the purpose of understanding the routing performed by the Integrated Endpoint process. For a detailed explanation of these fields, refer to the description of the DEST in appendix A.

Key Fields

The SOURCE CLASS and MESSAGE CLASS fields are the key fields the Destination object uses to search for DEST records. The Destination object compares transactions to the SOURCE CLASS and MESSAGE CLASS fields to determine which of the subsequent routing actions should be taken. The DESTINATION field is also part of the primary key to the DEST, but this field is not used to search the DEST.

SOURCE CLASS — A source class associated with a process or station from which a message can originate. This field contains a value that matches a SOURCECLASS or CLASS attribute value defined in a station or process definition in the XPNET system. The SOURCECLASS attribute is only available in systems running under XPNET 2.1 or greater. If both the SOURCECLASS and CLASS attributes are defined for a station or process, the value of the SOURCECLASS attribute is used. The SOURCECLASS attribute allows source class values up to four characters in length while the CLASS attribute allows source class values up to 16 characters in length.

MESSAGE CLASS — A series of four subfields used to determine how messages are routed. Specific values for these fields depend on the Endpoint Class Handler object being used.

Asterisks can be entered in one or more subfields if routing based on a specific subfield is not required. Asterisks are used with fifteen entries in the selection hierarchy when the Integrated Endpoint process searches the Destination File extended memory table. The selection hierarchy is described later in this section.

Action Fields

The DESTINATION, OBJECT NAME, TLO TO TLO LEXICON FLAG, and EPCH SERVICES TO OBJECT LEXICON fields specify the routing actions to take if a transaction matches on the SOURCE CLASS and MESSAGE CLASS key fields of the record. The Destination object can retrieve multiple DEST records to allow for several routing alternatives.

DESTINATION — The symbolic name of a process or station or the name of a service to which the transaction should be sent.

OBJECT NAME — The name of the message handler object within the destination process that will receive the message from this combination of source and message class and begin processing. For procedural destinations, this field should always be left blank.

TLO TO TLO LEXICON FLAG — A flag specifying whether the destination expects to receive incoming messages encapsulated within a Transport Layer Object to Transport Layer Object lexicon member function. For procedural destinations, this flag should always be set to a value of N.

EPCS SERVICES TO OBJECT LEXICON FLAG — A flag specifying whether the destination expects to receive incoming messages encapsulated within an Endpoint Class Handler Services to Object lexicon member function. For procedural destinations, this flag should always be set to a value of N. For ITS destinations, this flag is typically set to a value of Y if the TLO TO TLO LEXICON FLAG field is also set to a value of Y.

Selection Hierarchy

To select Destination File extended memory table entries, the Endpoint Class Handler object must supply the following information to the Destination object:

- The source class associated with the message
- The message class associated with the message
- The message class wildcard flag associated with the message source

Using the SOURCE CLASS field and the four subfields of the MESSAGE CLASS field, the Integrated Endpoint process searches through the key fields in the Destination File extended memory table entries looking for a match. In addition to an exact match, a wildcard character of any or all of the MESSAGE CLASS subfields is allowed. The hierarchy used for selection if the wildcard character (*) is used in any of the MESSAGE CLASS fields is described on the following page. To see how these subfields are used in processing, refer to the wildcard routing example provided later in this section and to the Destination object processing description provided in the *BASE24 IEP Objects Reference Manual*.

The message class wildcard flag configured for an application determines how the selection hierarchy is used. This flag identifies which MESSAGE CLASS subfields can use the wildcard character. The values of this flag correspond to each of the steps in the following selection hierarchy table. For the BASE24-telebanking application, the message class wildcard flag is set for each

interface instance in the MESSAGE CLASS WILDCARD OPTIONS fields on VRU Configuration Data (VCD) screen 2 and is stored in the ITS Interface Configuration File (IFCF). For other applications, this flag will be set in the appropriate application-specific configuration file.

The Endpoint Class Handler Services object includes the source class, message class, and message class wildcard values in a Message Route or Message Route Context Retrieve member function sent to the Endpoint Class Handler Services object. The Endpoint Class Handler Services object includes these values in the Destination Retrieve member function request sent to the Destination object.

Step	Field-1	Field-2	Field-3	Field-4
1	Value	Value	Value	Value
2	Value	Value	Value	**
3	Value	Value	**	Value
4	Value	Value	**	**
5	Value	**	Value	Value
6	Value	**	Value	**
7	Value	**	**	Value
8	Value	**	**	**
9	**	Value	Value	Value
10	**	Value	Value	**
11	**	Value	**	Value
12	**	Value	**	**
13	**	**	Value	Value
14	**	**	Value	**
15	**	**	**	Value
16	**	**	**	**

When the Destination object finds an entry that meets the above message class selection criteria, it adds the entry to the Destination Retrieve member function response.

Setting Up Message-Based Routing

The message class can be determined from any combination of message fields defined when the endpoint-specific Endpoint Class Handler object is developed. For current ITS applications developed by ACI, the message class is derived from the product ID (Field-1), the message type (Field-2 and Field-3), and the message variant (Field-4). All four fields consist of two bytes each. The product ID is a two-digit number specifying the product that originated the message. The message type is a four-digit ISO number indicating the type of message (for example, 0200—financial request, 0210—financial response, 0420—reversal, etc.). The message variant is a two-digit number that indicates the status of a given message (that is, 00—normal, 10—failed, 20—timeout, 30—reject, 40—return).

Message Class Distillation

Even if the message class is not derived directly from message fields, it can still be *distilled* from the message contents through the logic coded into the Endpoint Class Handler object. For example, messages with an application name of ATM could be given the value AM for Field-1, while messages with an application name of POS could be given the value PS for Field-1. In this case, Field-1 would be defined to contain the application name. Messages with a transaction type of “POS FINAN REQUEST” could be given the values 02 for Field-1, 02 for Field-2 and 00 for Field-3. In this case, Field-1 would contain a product identifier for POS (for example, 02 for POS transactions), Field-2 would contain the first two characters of the message type (for example, 02 for financial transactions), and Field-3 would contain the last two characters of the message type (for example, 00 for financial requests).

Considerations

The derivation of the message class is coded directly into the Endpoint Class Handler object for each endpoint serviced when the object is developed. When determining your message class requirements, you should consider where each message is coming from and where each message is going to. For each different destination, you should consider what distinguishes the message from going somewhere else. For using wildcard characters, you should consider what similar

but not identical messages going to the same destination share in common, and what parts are different. There are virtually no restrictions as to how the message class is derived.

Once you have determined how the message class is derived, the Destination File can be configured accordingly. All possible message classes that can be derived for your application, all possible message sources, and all possible message destinations should be accounted for in your Destination File configuration.

Note: For messages that are routed back to the transaction originating station or process, message-based routing is typically not required because the symbolic name of the originating station or process is usually carried in the original message.

Example Assumptions

The following pages provide examples of how the Destination object selects Destination File extended memory table entries for routing messages using different routing methods. These examples assume the message class derivation used for ACI-developed ITS applications, where:

- Field-1 of the MESSAGE CLASS field contains the two-character product ID
- Field-2 of the MESSAGE CLASS field contains the first two characters of the message type
- Field-3 of the MESSAGE CLASS field contains the last two characters of the message type
- Field-4 of the MESSAGE CLASS field contains the message variant.

Note: These examples only depict the message-based routing processing performed by the Integrated Endpoint process. Additional processing is performed by the Integrated Endpoint process, but it is not depicted in these examples.

Example Destination File Records

The routing method examples presented in this section use the DEST records shown below.

Note: These DEST records are for illustrative purposes only. They are not intended to represent a complete or recommended DEST configuration.

SOURCE CLASS	MESSAGE CLASS	DESTINATION	OBJECT NAME	TLO TO TLO LEXICON FLAG	EPCH SERVICES TO OBJECT LEXICON FLAG
AAA	01****00	APP^1	AAAT	Y	Y
BBB	010200**	APP^2		N	N
HOST	14023000	S1A^IVR		N	N
HYPR	*****	IASAUTH1	HYPR	Y	Y
HYPR	**0420**	P1A^REVERSE		N	N
IAS01	14021000	P1A^AUTH		N	N
MMM	15021000	APP^3	MMM	Y	Y
NCR	*****	IASAUTH1	NCR	Y	Y
IAS1	14023000	SIA^IVR		N	N
IAS2	14023000	S1A^IVR		N	N
ATH1	14011000	S1A^HOST		N	N
IVR	01010000	APPL1	VRUT	Y	Y
IVR	14010000	IASAUTH1	VRUT	Y	Y
IVR	14010000	IASAUTH2	VRUT	Y	Y
IVR	14022000	S1A^HOST1		N	N
IVR	14120000	P1A^IAS01	VRUT	Y	Y

SOURCE CLASS	MESSAGE CLASS	DESTINATION	OBJECT NAME	TLO TO TLO LEXICON FLAG	EPCH SERVICES TO OBJECT LEXICON FLAG
IVR	14220000	P1A^IAS01	VRUT	Y	Y
IVR	14220000	P1A^IAS02	VRUT	Y	Y
ZZZ	**0420**	REVERSE	ZZZ	Y	Y

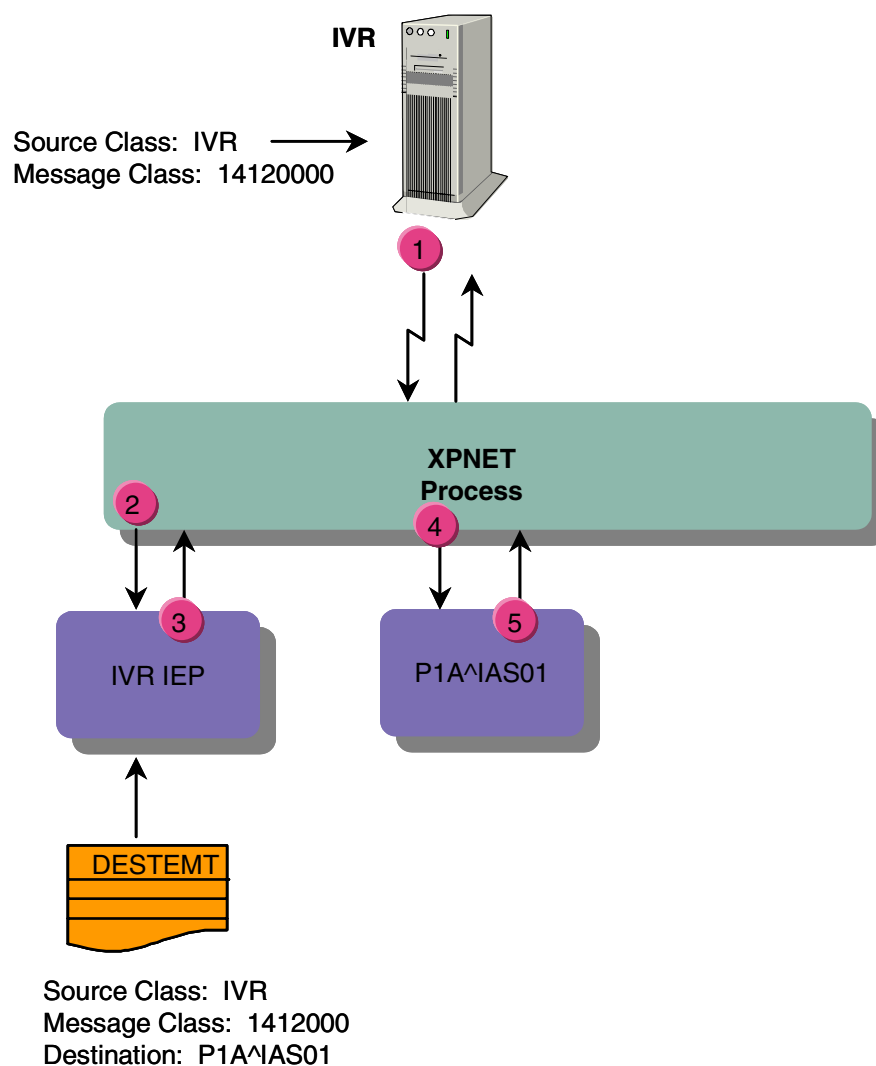
Simple Routing

Simple routing is used to route transactions from a station running under the XPNET process to an online application. The destination can be a procedural or ITS application.

In this example, a transaction request is routed from an integrated voice response system (IVR) to an Integrated Authorization Server process. The transaction response from the process is automatically routed back to the originating IVR for the corresponding request. Because the Integrated Endpoint process indicates that the response is to be routed back to the originating destination, no Destination File configuration is required to route the response.

If desired, the Destination File could be configured such that the response is sent to a destination other than the destination from which the request originated. If this were the case, another Destination File entry would be required where the source class would reflect the Integrated Authorization Server process as the message source.

Transaction flow processing for simple routing is illustrated on the following page. A key to the abbreviations follows the illustration. Flow step descriptions are provided following the key.



Key

DESTEMT = Destination File (DEST) extended memory table
 IAS = Integrated Authorization Server process
 IEP = Integrated Endpoint process
 IVR = Interactive Voice Response system

Processing is as follows:

1. An interactive voice response system (IVR) sends a transaction request to a station of the XPNET process.
2. The XPNET process delivers the request to the Integrated Endpoint process for this station.

3. The Integrated Endpoint process performs message-based routing as follows, determines the destination for the request to be the Integrated Authorization Server process, and sends the request to the XPNET process for delivery to the process.

The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object. Because the message class wildcard flag value is set to 0, only step 1 of the selection hierarchy table is used in the search of the Destination File extended memory table. Therefore, the Destination object searches only for an exact match in the Destination File extended memory table using the following combination of key values:

Source class	IVR			
Message class	14	12	00	00

The Destination object process steps through the key fields in the Destination File extended memory table sequentially as follows:

Step 1	IVR, 14120000, P1A^IAS01	Match found
--------	---------------------------------	-------------

The Destination object determines that messages with a source class of IVR and a message class of 14120000 are to be sent to the Integrated Authorization Server process (P1A^IAS01).

4. The XPNET process delivers the message to the Integrated Authorization Server process.
5. The Integrated Authorization Server process acquires the request, authorizes or denies the transaction, and routes the response back to the XPNET process for delivery to the IVR.
6. The XPNET process delivers the response back to the IVR.

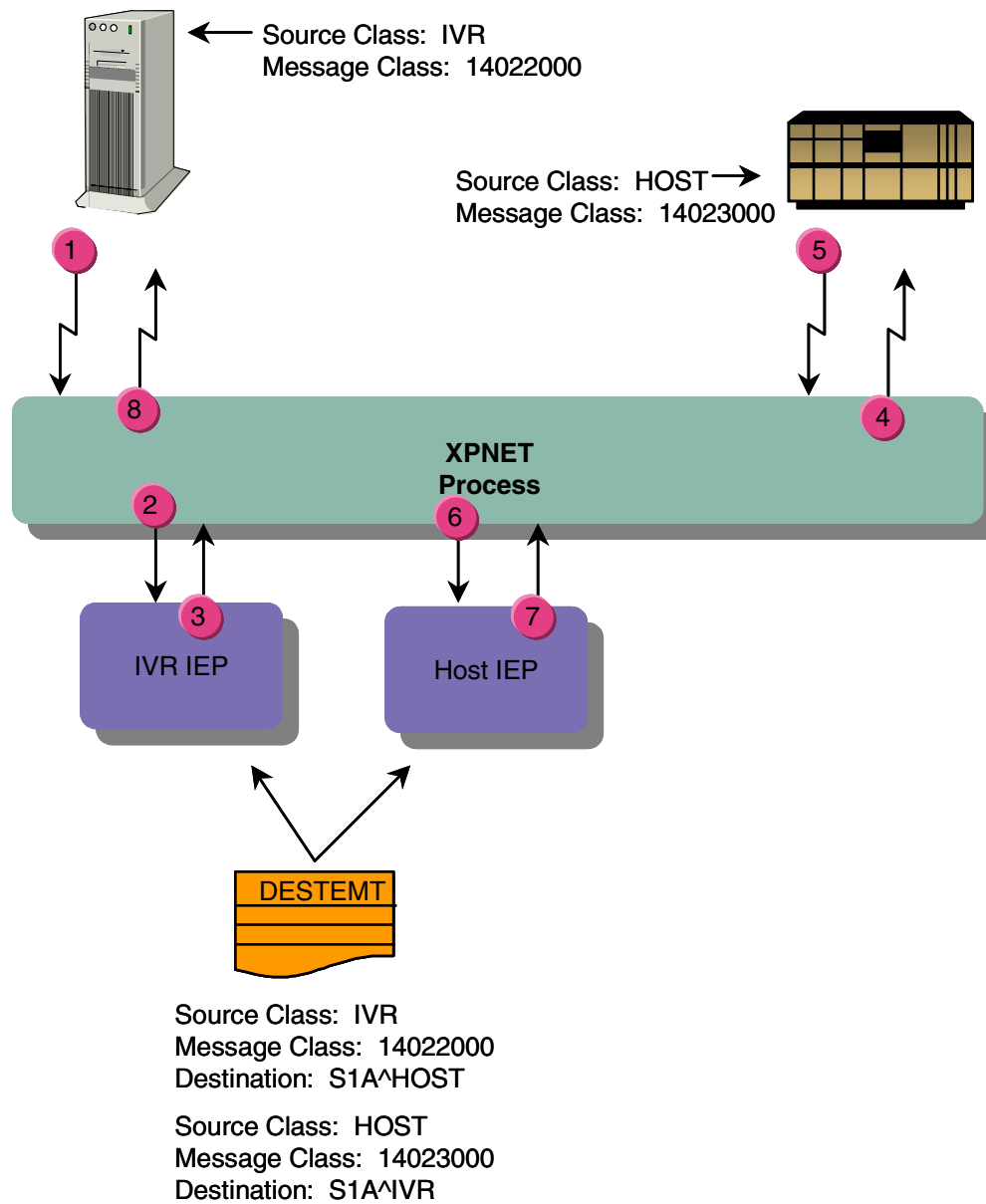
Pass-Through Routing

The Integrated Endpoint process allows messages to be routed from one station to another, without entering an application. This is called pass-through routing. With pass-through routing, no application processing takes place on the Tandem system; transactions are simply passed through the system.

In this example, messages with a message class of 14022000 received from an interactive voice response system (IVR) are passed to a host for authorization. No application processing takes place on the Tandem system. Instead, an application on the host authorizes the transaction.

Note: An additional destination needs to be configured to provide one-to-one station mapping for the response from the host, or the host must communicate with the Integrated Endpoint process in lexicons, providing the response station in a lexicon message. In this example, an additional DEST record is configured for the response.

Transaction flow processing for pass-through routing is illustrated on the following page. A key to the abbreviations follows the illustration. Flow step descriptions are provided following the key.



Key

DESTEMT = Destination File (DEST) extended memory table
 IEP = Integrated Endpoint process
 IVR = Interactive Voice Response system

Processing is as follows:

1. An interactive voice response system (IVR) sends a transaction request to a station of the XPNET process.
2. The XPNET process delivers the request to the Integrated Endpoint process for this station.
3. The Integrated Endpoint process performs message-based routing as follows, determines the destination for the request to be the host (S1A^HOST), and sends the request to the XPNET process for delivery to the host.

The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object. Because the message class wildcard flag value is set to 0, only step 1 of the selection hierarchy table is used in the search of the Destination File extended memory table. Therefore, the Destination object searches only for an exact match in the Destination File extended memory table using the following combination of key values:

Source class	IVR			
Message class	14	02	20	00

The Destination object process steps through the key fields in the Destination File extended memory table sequentially as follows:

Step 1	IVR, 14022000, S1A^HOST	Match found
--------	--------------------------------	-------------

The Destination object determines that messages with a source class of IVR and a message class of 14120000 are to be sent to the host (S1A^HOST).

4. The XPNET process delivers the message to the host for processing.
5. The host authorizes or denies the transaction and routes a response back to the XPNET process for delivery to the Integrated Endpoint process.
6. The XPNET process delivers the response to the Integrated Endpoint process.
7. The Integrated Endpoint process performs message-based routing as follows, determines the destination for the request to be the IVR (S1A^IVR), and sends the request to the XPNET process for delivery to the IVR.

The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object. Because the message class wildcard flag value is set to 0, only step 1 of the selection hierarchy table is used in the search of the Destination File extended memory table. Therefore, the Destination object searches only for an exact match in the Destination File extended memory table using the following combination of key values:

Source class	HOST			
Message class	14	02	30	00

The Destination object process steps through the key fields in the Destination File extended memory table sequentially as follows:

Step 1

HOST, 14023000, S1A^IVR

Match found

The Destination object determines that messages with a source class of HOST and a message class of 14023000 are to be sent to the IVR (S1A^IVR).

8. The XPNET process delivers the transaction response to the IVR.

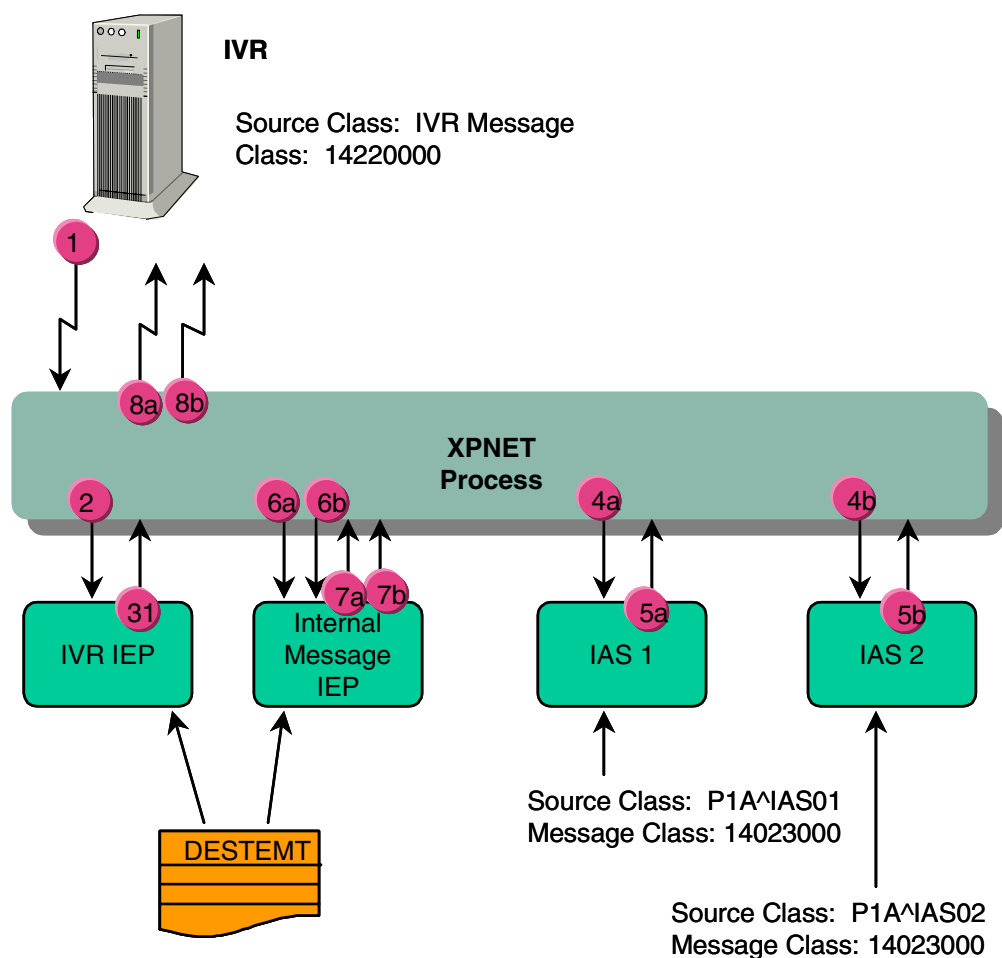
Broadcast Routing

The Integrated Endpoint process allows messages to be routed simultaneously to more than one destination in parallel. This is called broadcast routing. Broadcast routing can be used when simultaneous processing of a transaction is required by more than one application.

Note: The Integrated Endpoint process does not coordinate responses from broadcast messages. This is an application responsibility. Both the source class and the message class are typically changed on a response, but not always.

In this example, incoming transactions for the message class of 14022000 received from an interactive voice response system (IVR) are simultaneously routed to two different application servers. Both destinations are assumed to be ITS applications.

Transaction flow processing for broadcast routing is illustrated on the following page. A key to the abbreviations follows the illustration. Flow step descriptions are provided following the key.



Source Class: IVR	Source Class: P1A^IAS01
Message Class: 14220000	Message Class: 14023000
Destination: P1A^IAS01	Destination: S1A^IVR
Source Class: IVR	Source Class: P1A^IAS02
Message Class: 14220000	Message Class: 14023000
Destination: P1A^IAS02	Destination: S1A^IVR

Key

DESTENT = Destination File (DEST) extended memory table
IAS = Integrated Authorization Server
IEP = Integrated Endpoint process
IVR = Interactive Voice Response system

Processing is as follows:

1. An interactive voice response system (IVR) sends a transaction request to a station of the XPNET process.
2. The XPNET process delivers the request to the Integrated Endpoint process for this station.
3. The Integrated Endpoint process performs message-based routing as follows, determines the destinations for the request to be the P1A^IAS01 process and the P1A^IAS02 process, and sends the request to the XPNET process for delivery to each of the specified destinations.

The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object. Because the message class wildcard flag value is set to 0, only step 1 of the selection hierarchy table is used in the search of the Destination File extended memory table. Therefore, the Destination object searches only for an exact match in the Destination File extended memory table using the following combination of key values:

Source class	IVR			
Message class	14	22	00	00

The Destination object process steps through the key fields in the Destination File extended memory table sequentially as follows:

Step 1	IVR, 14220000, P1A^IAS01	Match found
Step 1	IVR, 14220000, P1A^IAS02	Match found

The Destination object determines that messages with a source class of IVR and a message class of 14220000 are to be sent simultaneously to the P1A^IAS01 and P1A^IAS02 processes.

4. The XPNET process delivers one of the request messages to the P1A^IAS01 process and the other to the P1A^IAS02 process.
 - a. The XPNET process delivers the request message to the P1A^IAS01 process.
 - b. The XPNET process delivers the request message to the P1A^IAS02 process.

5. Both Integrated Authorization Server processes process the request, generate responses, and send the responses to the XPNET process for delivery to the Integrated Endpoint process for messages received from an internal process.
 - a. The P1A^IAS01 process generates a response and forwards it to the XPNET process for delivery.
 - b. The P1A^IAS02 process generates a response and forwards it to the XPNET process for delivery.
6. The XPNET process delivers the response messages to the Internal Message Integrated Endpoint process.
 - a. The XPNET process delivers the response from the P1A^IAS01 process to the Internal Message Integrated Endpoint process.
 - b. The XPNET process delivers the response from the P1A^IAS02 process to the Internal Message Integrated Endpoint process.
7. The Internal Message Integrated Endpoint process performs message-based routing for each transaction response received, determines that the destination for both messages is the S1A^IVR station, and sends each response to the XPNET process for delivery to the station. Each response is handled separately.
 - a. The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object for the message received from the P1A^IAS01 process. Because the message class wildcard flag value is set to 0, only step 1 of the selection hierarchy table is used in the search of the Destination File extended memory table. Therefore, the Destination object searches for an exact match in the Destination File extended memory table using the following combination of key values:

Source class	IAS01
Message class	14 02 30 00

The Destination object process steps through the key fields in the Destination File extended memory table sequentially as follows:

Step 1	IAS01, 14023000, S1A^IVR	Match found
--------	---------------------------------	-------------

The Destination object determines that messages with a source class of IAS01 and a message class of 14023000 are to be sent to the S1A^IVR station.

The response from the P1A^IAS01 process is sent to the XPNET process for delivery to the IVR station.

- b. The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object for the message received from the P1A^IAS02 process. Because the message class wildcard flag value is set to 0, only step 1 of the selection hierarchy table is used in the search of the Destination File extended memory table. Therefore, the Destination object searches for an exact match in the Destination File extended memory table using the following combination of key values:

Source class	IAS02			
Message class	14	02	30	00

The Destination object process steps through the key fields in the Destination File extended memory table sequentially as follows:

Step 1

IAS02, 14023000, S1A^IVR

Match found

The Destination object determines that messages with a source class of IAS02 and a message class of 14023000 are to be sent to the S1A^IVR station.

The response from the P1A^IAS02 process is sent to the XPNET process for delivery to the IVR station.

8. The IVR station delivers the response messages to the IVR.
 - a. The IVR station delivers the response from the P1A^IAS01 process to the IVR.
 - b. The IVR station delivers the response from the P1A^IAS02 process to the IVR.

Chained Routing

The Integrated Endpoint process allows routing to more than one destination in series. This is called chained routing. In chained routing, the applications that handle the message can have different source classes and can modify the message class.

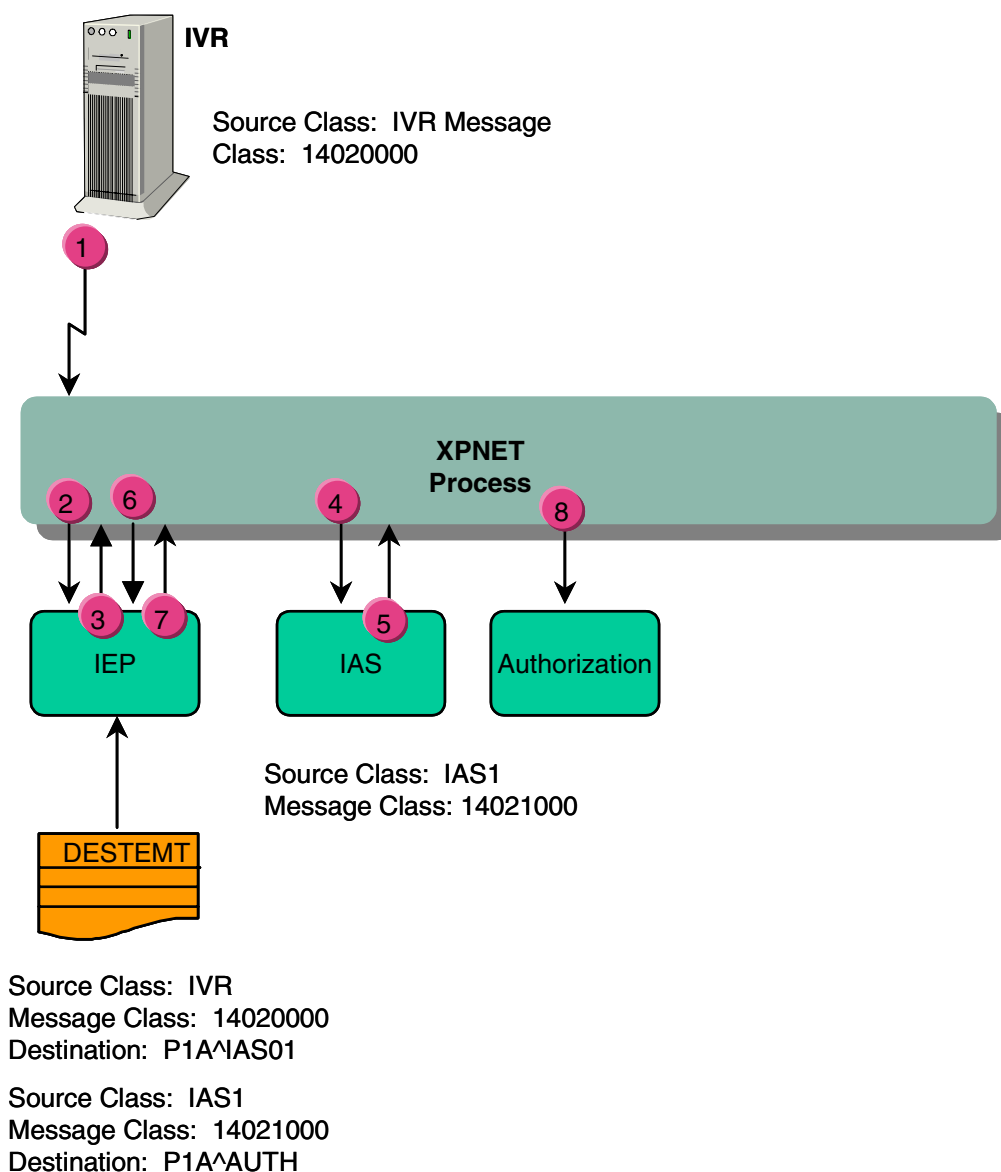
In this example, transactions received from an Interactive Voice Response system (IVR) with a message class of 14120000 are routed to the P1A^IAS01 application process. When the P1A^IAS01 process has completed processing, a transaction response is returned to the Integrated Endpoint process with the source class changed to IAS1. This response is then forwarded to yet another application server for further processing. In this case, messages with a source class of IAS1 and a message class of 14021000 are routed to the P1A^AUTH process. Chaining can be configured for as many links as are needed in the processing chain. A different Destination File record must be configured for each link in the chain.

For each link in the chain, either the source class or the message class is changed. Therefore, a Destination File configuration record is required for the routing of transaction messages for each link in the chain.

In this example, the first destination is assumed to be an ITS process while the second is assumed to be a procedural process.

Transaction flow processing for chained routing is illustrated on the following page. A key to the abbreviations follows the illustration. Flow step descriptions are provided following the key.

Note: For clarification, only the request portion of a transaction is shown. If a response message were to be routed back through the chain, two additional Destination File records would be required, with a record for each link in the processing chain.



Key

DESTENT	= Destination File (DEST) extended memory table
IAS	= Integrated Authorization Server
IEP	= Integrated Endpoint process
IVR	= Interactive Voice Response system

Processing is as follows:

1. An interactive voice response system (IVR) sends a transaction request to a station of the XPNET process.
2. The XPNET process delivers the request to the Integrated Endpoint process for this station.
3. The Integrated Endpoint process performs message-based routing as follows, determines the destination for the request to be the P1A^IAS01 process, and sends the request to the XPNET process for delivery to the specified destination.

The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object for the message received from the S1A^IVR station. Because the message class wildcard flag value is set to 0, only step 1 of the selection hierarchy table is used in the search of the Destination File extended memory table. Therefore, the Destination object searches for an exact match in the Destination File extended memory table using the following combination of key values:

Source class	IVR			
Message class	14	12	00	00

The Destination object process steps through the key fields in the Destination File extended memory table sequentially as follows:

Step 1	IVR, 14120000, P1A^IAS01	Match found
--------	---------------------------------	-------------

The Destination object determines that messages with a source class of IVR and a message class of 14120000 are to be sent to the P1A^IAS01 process destination.

The request from the IVR station is sent to the XPNET process for delivery to the P1A^IAS01 process.

4. The XPNET process delivers the message to the P1A^IAS01 authorization server process.

The P1A^IAS01 process processes the message and sends a message back to the Integrated Endpoint process for additional processing. In this message, the source class for the message is changed to IAS1 and the message class is changed to 14021000.

5. The IAS process delivers the message to the XPNET process.
6. The XPNET process delivers the message back to the Integrated Endpoint process.

7. The Integrated Endpoint process performs message-based routing as follows, determines the destination for the request to be the P1A^AUTH process, and sends the request to the XPNET process for delivery to the specified destination.

The Endpoint Class Handler Services sends the Destination Retrieve member function to the Destination object. Because the message class wildcard flag value is set to 0, only step 1 of the selection hierarchy table is used in the search of the Destination File extended memory table. Therefore, the Destination object searches for an exact match in the Destination File extended memory table using the following combination of key values:

Source class	IAS1			
Message class	14	02	10	00

The Destination object process steps through the key fields in the Destination File extended memory table sequentially as follows:

Step 1	IAS1, 14021000, P1A^AUTH	Match found
--------	---------------------------------	-------------

The Destination object determines that messages with a source class of IAS1 and a message class of 14021000 are to be sent to the P1A^AUTH process destination.

The request from the P1A^IAS01 process is sent to the XPNET process for delivery to the P1A^AUTH process.

8. The XPNET process delivers the message to the P1A^AUTH process for further processing.

Combined Broadcast and Chained Routing

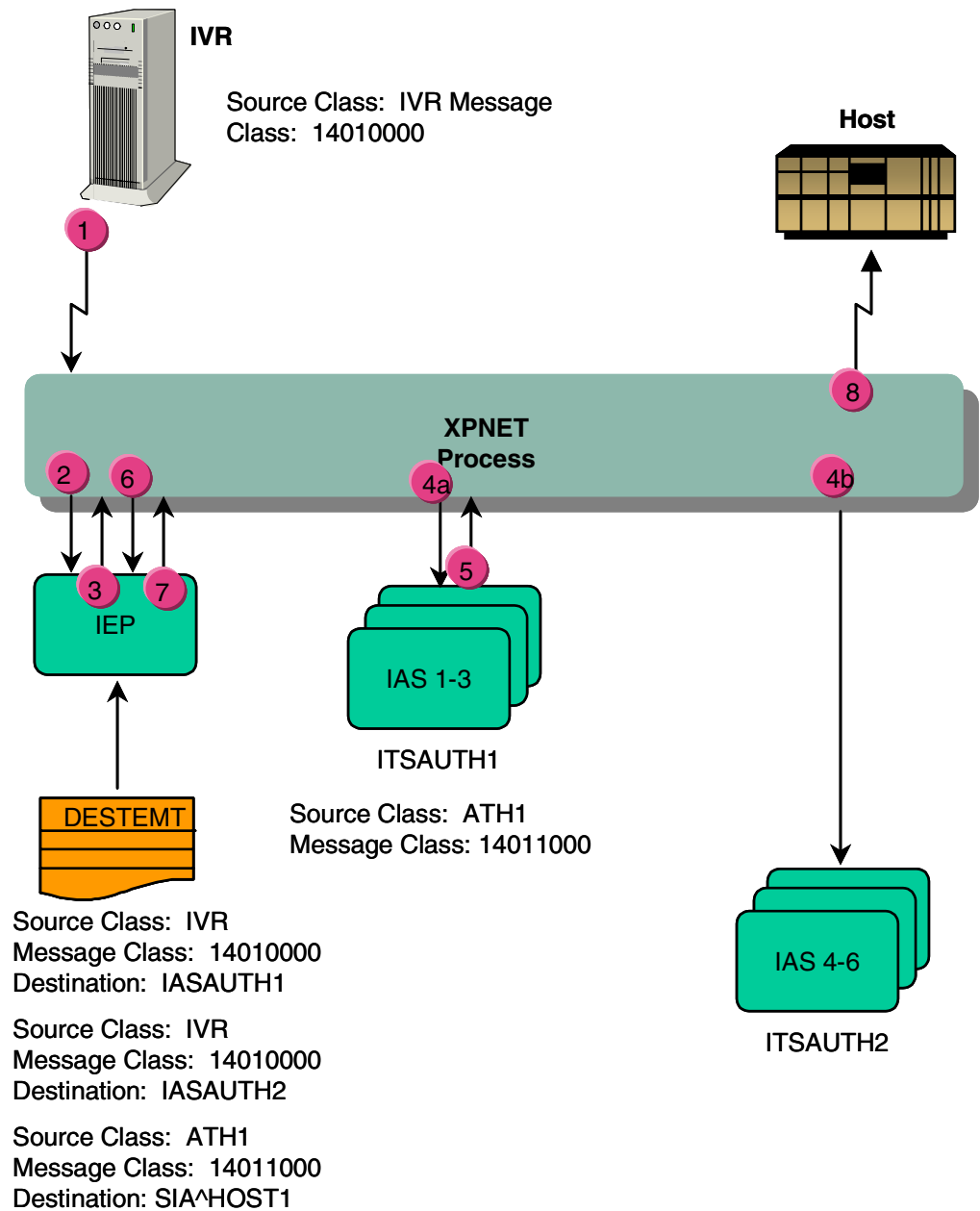
The Integrated Endpoint process allows for a combination of both broadcast and chained routing to be used. This configuration provides the most flexibility in transaction routing. Messages can be broadcast to multiple applications while also being chain routed from one application to another.

In this example, transactions received from an Interactive Voice Response system (IVR) with a message class of 14010000 are simultaneously broadcast to the services for application 1 (IASAUTH1) and application 2 (IASAUTH2). When the transaction response is received back from application 1 with the source class changed to ATH1 and the message class changed to 14011000, it is routed to a host (S1A^HOST). Both the IASAUTH1 and IASAUTH2 application destinations are assumed to be services of ITS processes. The host is assumed to be procedural-based.

Transaction flow processing for combined broadcast and chained routing is illustrated on the following page. A key to the abbreviations follows the illustration. Flow step descriptions are provided following the key.

Note: The Integrated Endpoint process does not wait for both messages that are broadcast to be returned before sending the message from the IASAUTH1 application on to the host. As soon as a message is returned from the IASAUTH1 application it is chain routed to the host.

For clarification, the response portions of the transactions from the final destinations (that is, IASAUTH2 and S1A^HOST) are not depicted.



Key

DESTEMT	= Destination File (DEST) extended memory table
IAS	= Integrated Authorization Server
IEP	= Integrated Endpoint process
IVR	= Interactive Voice Response system

Processing is as follows:

1. An interactive voice response system (IVR) sends a transaction request to a station of the XPNET process.
2. The XPNET process delivers the request to the Integrated Endpoint process for this station.
3. The Integrated Endpoint process performs message-based routing as follows, determines the destinations for the request to be the IASAUTH1 and IASAUTH2 services, and sends the request to the XPNET process for delivery to both of the specified destinations.

The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object for the message received from the S1A^IVR station. Because the message class wildcard flag value is set to 0, only step 1 of the selection hierarchy table is used in the search of the Destination File extended memory table. Therefore, the Destination object searches only for an exact match in the Destination File extended memory table using the following combination of key values:

Source class	IVR
Message class	14 01 00 00

The Destination object process steps through the key fields in the Destination File extended memory table and finds matches on the following records:

Step 1	IVR, 14010000, IASAUTH1	Match found
Step 1	IVR, 14010000, IASAUTH2	Match found

The Destination object determines that messages with a source class of IVR and a message class of 14010000 are to be sent simultaneously to both the IASAUTH1 and IASAUTH2 service destinations.

The request from the S1A^IVR station is sent to the XPNET process for delivery to the IASAUTH1 and IASAUTH2 services.

4. The XPNET process delivers the message to the IASAUTH1 and the IASAUTH2 services.
 - a. The XPNET process delivers the message to the IASAUTH1 service where the message is acquired by the P1A^IAS01 process.
 - b. The XPNET process delivers the message to the IASAUTH2 service where the message is acquired by the P1A^IAS04 process.

5. The P1A^IAS01 process in the IASAUTH1 service authorizes the transaction. The NEF indicates a new source class for the message and the process changes the message class. The P1A^IAS01 process sends the message back to the Integrated Endpoint process for additional routing using the XPNET process.
6. The XPNET process delivers the response message from the P1A^IAS01 process to the Integrated Endpoint process.
7. The Integrated Endpoint process performs message-based routing as follows, determines the destination for the request to be the S1A^HOST1 station, and sends the request to the XPNET process for delivery to both of the specified destination.

The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object for the message received from the P1A^IAS01 process. Because the message class wildcard flag value is set to 0, only step 1 of the selection hierarchy table is used in the search of the Destination File extended memory table. Therefore, the Destination object searches for an exact match in the Destination File extended memory table using the following combination of key values:

Source class	ATH1
Message class	14 01 10 00

The Destination object process steps through the key fields in the Destination File extended memory table and finds a match on the following record:

Step 1	ATH1, 14011000, S1A^HOST1	Match found
--------	----------------------------------	-------------

The Destination object determines that messages with a source class of ATH1 and a message class of 14011000 are to be sent to the S1A^HOST station destination.

The request from the P1A^IAS01 process is sent to the XPNET process for delivery to the S1A^HOST1 station.

8. The XPNET process delivers the message to the host.

Wildcard Routing

For any of the message-based routing scenarios previously presented in this section, wildcard routing of some or all of the message class is allowed using the wildcard character (*). Wildcard routing can be as specific or as general as desired, with the more specific cases always searched for and used first. For example, if you configured the Destination File with the wildcard character in all of the message class subfields, but also have a particular case where the routing is specific to one or more of the message class subfields, the more specific case will always be used before the more general case.

Wildcard routing can be used to route all messages received, no matter what type, from a particular source to the same application process or service. In such a case all messages received from automated teller machine (ATM) devices could be routed to an ATM application process, while all messages received from point-of-sale (POS) devices could be routed to a POS service, regardless of specific message types.

On the other hand, you can manipulate the message class wildcard character so that all messages except for a specific type of message are sent to one destination while transactions of a specific message type are directed to a different destination.

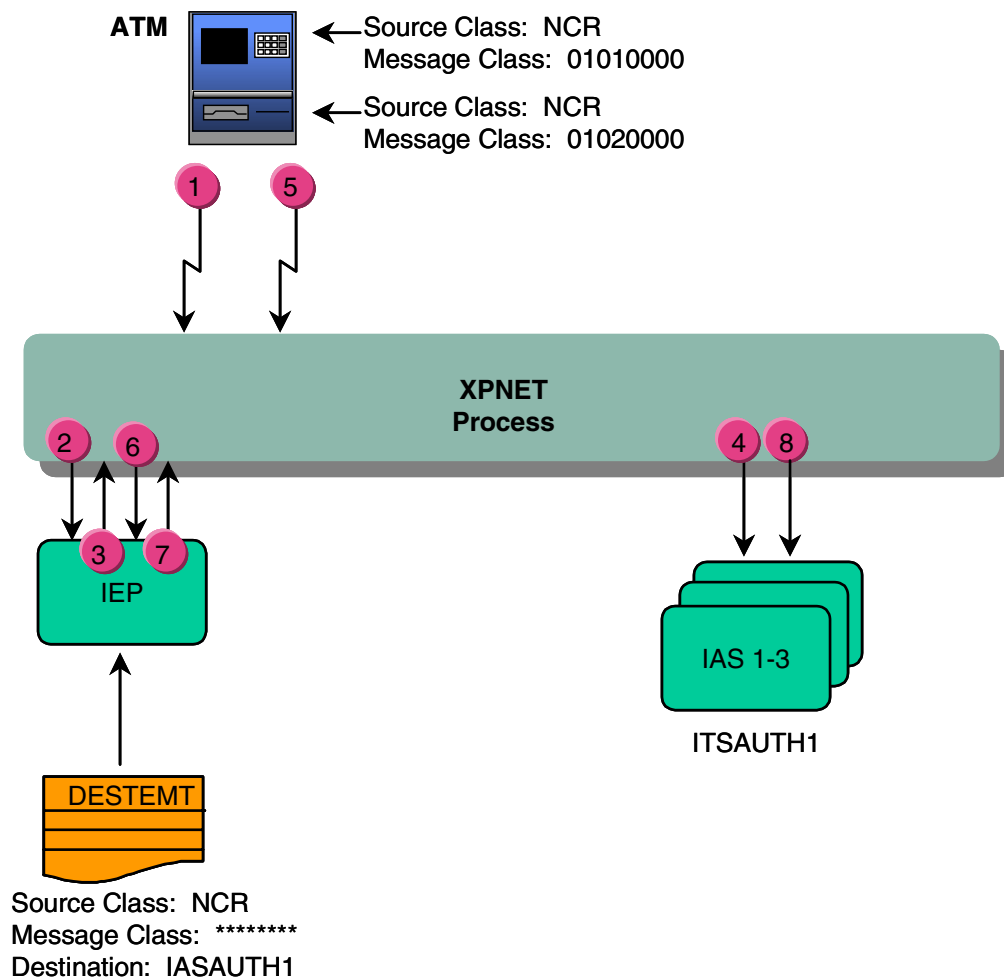
In the following wildcard examples, a wildcard character is used for all or some of the message class subfields.

Note: For clarification, the routing for the transaction responses is not shown. In these examples, message-based routing would only need to be configured for the responses if they are routed back to a destination other than the originating station.

Routing Example 1

In the first example, all transactions received from an NCR95xx series ATM device are routed to the same ITS application service, regardless of specific message types. The destination service is composed of Integrated Server processes.

Transaction flow processing for this wildcard routing example is illustrated on the following page. A key to the abbreviations follows the illustration. Flow step descriptions are provided following the key.



Key

ATM	= Automatic Teller Machine
DESTEMT	= Destination File (DEST) extended memory table
IAS	= Integrated Authorization Server
IEP	= Integrated Endpoint process

Processing is as follows:

1. An NCR95xx series automatic teller machine (ATM) sends a transaction request to a station of the XPNET process.
2. The XPNET process delivers the request to the Integrated Endpoint process for this station.

3. The Integrated Endpoint process performs message-based routing as follows, determines the destination for the request to be the IASAUTH1 service, and sends the request to the XPNET process for delivery to both of the specified destination.

The Endpoint Class Handler object determines the message class to be 01010000. The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object for the message received from the S1A^NCR95 station. Because the message class wildcard flag value indicates that all characters of the message class subfields use the wildcard character, only step 16 of the selection hierarchy table provided earlier in this section is used. Therefore, the Destination object searches for all Destination File records matching the source class only using the following combination of key values:

Source class	NCR				
Message class	**	**	**	**	

The Destination object process steps through the key fields in the Destination File extended memory table and finds only one match on the following record:

Step 16	NCR, *****, IASAUTH1	Match found
---------	-----------------------------	-------------

The Destination object determines that all messages with a source class of NCR and any message class are always to be sent to the IASAUTH1 service.

The request from the S1A^NCR95 station is sent to the XPNET process for delivery to the IASAUTH1 service.

4. The XPNET process delivers the message to the IASAUTH1 service where it is acquired by an available Integrated Authorization Server process within the service.
5. The NCR95xx series ATM sends a second transaction request to a station of the XPNET process. This request has a different message class.
6. The XPNET process delivers the request to the Integrated Endpoint process for this station.
7. The Integrated Endpoint process performs message-based routing as follows, determines the destination for the request to be the IASAUTH1 service, and sends the request to the XPNET process for delivery to the specified destination.

The Endpoint Class Handler object determines the message class to be 01020000. The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object for the message received from the S1A^NCR95 station. Because the message class wildcard flag value

indicates that all characters of the message class subfields use the wildcard character, only step 16 of the selection hierarchy table provided earlier in this section is used. Therefore, the Destination object searches for all Destination File records matching the source class only using the following combination of key values:

Source class	NCR				
Message class	**	**	**	**	**

The Destination object process steps through the key fields in the Destination File extended memory table and finds only one match on the following record:

Step 16	NCR, *****, IASAUTH1	Match found
---------	-----------------------------	-------------

The Destination object determines that all messages with a source class of NCR and any message class are always to be sent to the IASAUTH1 service.

The second request from the S1A^NCR95 station is sent to the XPNET process for delivery to the IASAUTH1 service.

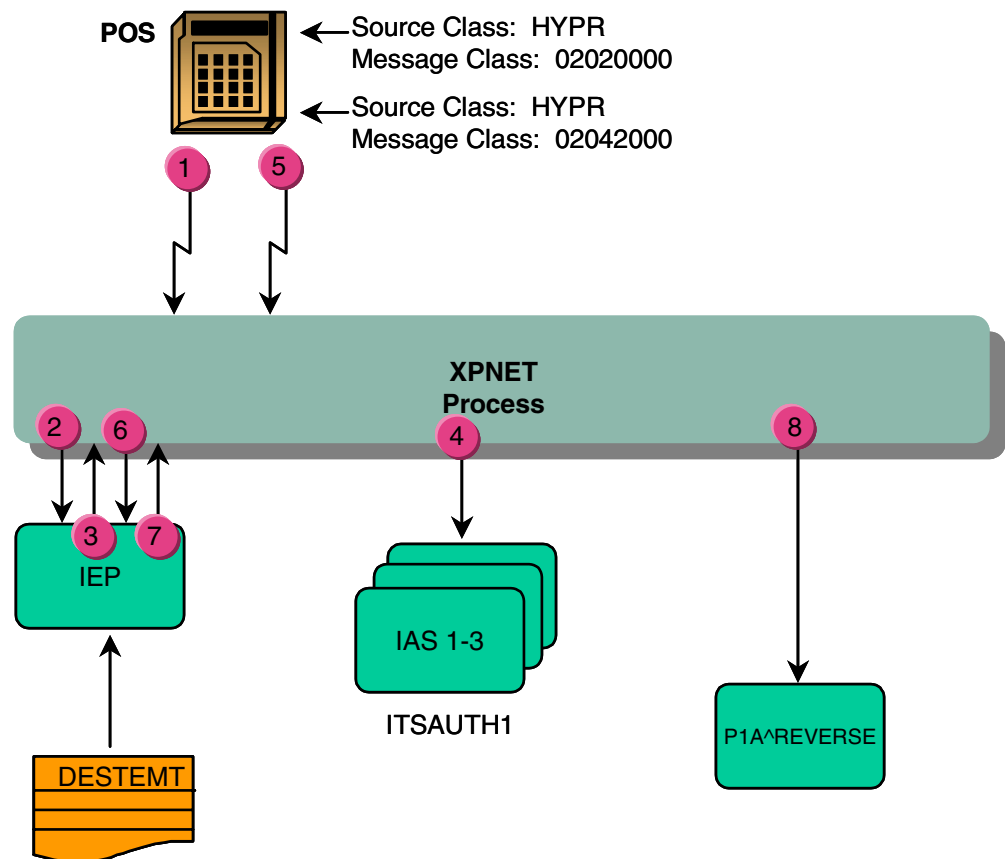
8. The XPNET process delivers the second message to the IASAUTH1 service where it is acquired by an available Integrated Authorization Server process within the service.

Routing Example 2

In this second wildcard routing example, all messages received from a Hypercom POS device are routed to a single ITS destination, except for cardholder reversal transactions, which are routed to a separate procedural application process. In this scenario it is assumed that POS cardholder reversal transactions are identified with a message type of 0420. In this example it is also assumed that the second and third message class subfields contain the message type.

The destination service is composed of Integrated Server processes, while the destination process for reversal transactions is procedural.

Transaction flow processing for this wildcard routing example is illustrated on the following page. A key to the abbreviations follows the illustration. Flow step descriptions are provided following the key.



Source Class: HYPR
Message Class: *****
Destination: IASAUTH1

Source Class: HYPR
Message Class: **0420**
Destination: P1A^REVERSE

Key

DESTENT = Destination File (DEST) extended memory table
IAS = Integrated Authorization Server
IEP = Integrated Endpoint process
POS = Point-of-Sale device

Processing is as follows:

1. A Hypercom point-of-sale (POS) device sends a financial transaction request to a station of the XPNET process.
2. The XPNET process delivers the request to the Integrated Endpoint process for this station.
3. The Integrated Endpoint process performs message-based routing as follows, determines the destination for the request to be the IASAUTH1 service, and sends the request to the XPNET process for delivery to the specified destination.

The Endpoint Class Handler object determines the message class to be 02020000. The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object for the message received from the S1A^HYPER station. Because the message class wildcard flag value is set to 9 indicating that Field-1 and Field-4 of the message class subfields use the wildcard character, steps 10, 12, 14, and 16 of the selection hierarchy table provided earlier in this section are used. Therefore, the Destination object searches for all Destination File records matching the source class and the following combinations of message class key values:

Source class	HYPR			
Message class	**	04	20	**
	**	04	**	**
	**	**	20	**
	**	**	**	**

The Destination object process steps through the key fields in the Destination File extended memory table and finds only one match on the following record:

Step 10	HYPR, **0420**, P1A^REVERSE	No match
Step 12	HYPR, **04****	No match
Step 14	HYPR, ****20**	No match
Step 16	HYPR, *****, IASAUTH1	Match found

The Destination object determines that all messages with a source class of HYPR and a message type of 0200 are always to be sent to the IASAUTH1 service.

The request from the S1A^HYPER station is sent to the XPNET process for delivery to the IASAUTH1 service.

4. The XPNET process delivers the financial request message to the IASAUTH1 service where it is acquired by an available Integrated Authorization Server process within the service.
5. The Hypercom POS device sends a financial reversal transaction request to a station of the XPNET process.
6. The XPNET process delivers the request to the Integrated Endpoint process for this station.
7. The Integrated Endpoint process performs message-based routing as follows, determines the destination for the request to be the P1A^REVERSE process, and sends the request to the XPNET process for delivery to both of the specified destination.

The Endpoint Class Handler object determines the message class to be 02042000. The Endpoint Class Handler Services object sends the Destination Retrieve member function to the Destination object for the message received from the S1A^HYPER station. Because the message class wildcard flag value is set to 9 indicating that Field-1 and Field-4 of the message class subfields use the wildcard character, steps 10, 12, 14, and 16 of the selection hierarchy table provided earlier in this section are used. Therefore, the Destination object searches for all Destination File records matching the source class and the following combinations of message class key values:

Source class	HYPR			
Message class	**	04	20	**
	**	04	**	**
	**	**	20	**
	**	**	**	**

The Destination object process steps through the key fields in the Destination File extended memory table and immediately finds a match on the following record. The remaining combinations of message class values are not checked.

Step 10	HYPR, **0420**, P1A^REVERSE	Match found
Step 12	HYPR, **04****	Not checked
Step 14	HYPR, *****20**	Not checked
Step 16	HYPR, *****, IASAUTH1	Not checked

The Destination object determines that all messages with a source class of HYPR and a message type of 0420 are always sent to the P1A^REVERSE process.

The second request from the S1A^HYPER station is sent to the XPNET process for delivery to the P1A^REVERSE process.

8. The XPNET process delivers the message to the P1A^REVERSE process.

Section 4

Measure Counter Configuration and Reporting

This section describes the steps required to implement Measure user counter updating for the Integrated Endpoint process and how to view and report on Measure counters. The use of Measure user counters with the Integrated Endpoint process is optional.

For detailed information on the Measure command interface (MEASCOM) and any of its commands used in this section, refer to the following Tandem publications:

- *Measure Reference Manual*
- *Measure User's Guide*

Measure Counters

The Integrated Endpoint process uses the Tandem Measure product to track various Integrated Endpoint processing statistics. The Endpoint Class Handler and Endpoint Class Handler Services objects maintain the processing statistics for the Integrated Endpoint process.

Measure counters are updated by the Endpoint Class Handler Services object for all financial responses, financial advice and reversal responses, timeouts, and failed messages processed by the Integrated Endpoint process.

Each time a Measure counter is updated, the Endpoint Class Handler Services object passes several parameters to Measure, including two time parameters representing the authorization time and the internal time as well as a count of the number of messages processed.

The endpoint-specific Endpoint Class Handler object updates the same type of Measure user counters for all late response messages received.

The failed message count statistic is tracked for all messages.

The following statistics are tracked for all authorization request and response messages:

- External authorization count
- External authorization time
- Internal authorization count
- Internal authorization time
- Authorization count
- Authorization time
- Timeout count

The following statistics are tracked for financial advice and reversal advice messages:

- External authorization count
- External authorization time
- Internal authorization count

- Internal authorization time
- Timeout count

Instructions for configuring user counters to Measure, starting a Measure session, and viewing Measure results are provided later in this section. Refer to the ***BASE24 IEP Objects Reference Manual*** for details on how the Endpoint Class Handler object and the Endpoint Class Handler Services object implement Measure user counters.

Note: Transaction rates are not included in the Measure user counters maintained by the Integrated Endpoint process. To determine transaction rates, you should use either a Distributed System Management Application (DSMA) or the NET24 Performance Monitoring Subsystem. For more information on the NET24 Performance Monitoring Subsystem, refer to the ***NET24 Performance Monitoring Manual***.

Configuration

The following paragraphs discuss the configuration requirements for implementing Measure counter logic in the Integrated Endpoint process.

Configuration Steps

You must perform the following steps to implement Measure user counter accumulation in the Integrated Endpoint process.

1. Define a Measure user counter name for each entity for which statistics are to be collected by the Integrated Endpoint process for your application. For example, if statistics are to be collected on an endpoint basis, a Measure user counter name must be defined for each endpoint. Statistics cannot be gathered across endpoints because each endpoint is processed by a unique Endpoint Class Handler object.

The statistics for each endpoint (for example, interactive voice response system, host, etc.) must be handled separately. If statistics are to be collected on an interface instance basis, a Measure user counter name must be defined for each interface instance for an endpoint. Each interface instance provides a means of dividing up interface processing according to specific business relationships and purposes. For example, you could have a separate interface instance for each financial institution using the interface. If statistics are to be collected on an endpoint basis only, the same user counter name should be used for all interface instances to that endpoint. Some applications may require that the Measure user names you define be configured to the application. The actual means by which Measure user counter names are established to an application are application-specific and are beyond the scope of this manual.

Note: The following information is specific to the BASE24-telebanking application only and is presented as an example of how statistics can be configured to be maintained at an interface instance level. The actual level at which statistics are maintained for your application may differ from that presented here due to the specific business requirements of your application. If you are not using the BASE24-telebanking application, this information applies only as an example of the level at which statistics can be maintained.

For each interface instance of an Integrated Endpoint process used in your system, fill in the appropriate fields on the BASE24-telebanking VRU Configuration Data (VCD) screen and add a record.

The MEASURE COUNTER NAME field on the VCD screen must be configured for each interface instance for which processing statistics are to be maintained. If this field is not configured for an interface instance, then statistics are not kept for that interface instance.

Note: You can use the same name across ITS Interface Configuration File (IFCF) records to group statistics into one counter or you can make each name unique to gather statistics about each interface separately. The grouping you choose is dependent upon your specific business requirements for the BASE24-telebanking application.

The MEASURE COUNTER NAME field defines the name of the user counter used by the Tandem Measure product to accumulate statistics on behalf of each interface instance defined.

As an example, you could have one interface instance for the entire IVR interface, or you could divide it up by having a separate interface instance for each financial institution using the IVR interface. Every record in the Interface Configuration File (IFCF) defines a separate interface instance.

For detailed information on the MEASURE COUNTER NAME field and the VCD screen, refer to the *BASE24 Core Files and Tables Maintenance Manual*.

2. Add each of the Measure user counter names established in step 1 to the Measure subsystem.

Note: If you are using the BASE24-telebanking application, a Measure user counter name must be defined for each MEASURE COUNTER NAME value defined on the VCD screen in step 1. If you are not using the BASE24-telebanking application, a Measure user counter name must be configured for each Measure counter name defined in step 1.

The following commands add a user counter to the Measure subsystem.

- a. Log on to the Tandem computer and start a Measure command interface session by entering the following command at the TACL prompt:

```
TACL> MEASCOM
```

- b. Enter the following commands to add each user counter, where *counter-name* is the user name you assign to the Measure counter and *\$vol* is the volume where the Integrated Endpoint process executable object code is located. You must first specify the Integrated Endpoint process executable object code as the target for the ADD COUNTER commands using the ADD USERDEF command.

Note: If you have more than one endpoint in your system, there may be a separate Integrated Endpoint process executable object file for each endpoint.

```
1+ ADD USERDEF $vol.OE13OBJ.IEPLO
2+ ADD COUNTER counter-name, PROCESS $vol.OE13OBJ.IEPLO,
    ACCUM, ARRAY 15
```

Because the same ADD COUNTER commands must be specified when you start a measurement, you may want to place these commands in an obey file for subsequent use.

The user counter is set to ACCUM to establish this Measure counter as an accumulating type of counter, even though some of the individual counters measured in the array are really queue counters. The queue type counters are still handled correctly.

The array is set to 15 because there are 15 individual measurements maintained by the Endpoint Class Handler Services object for each Measure user counter name. Each of these individual measurements is assigned an index in the array from 0 to 14 for the Endpoint Class Handler Services object as described in the ***BASE24 IEP Objects Reference Manual***.

NET24 Performance Monitoring Subsystem

If you intend to use the NET24 Performance Monitoring Subsystem to monitor Integrated Endpoint processing rates, additional Measure user counters with a counter type of transaction rate must be established for each Integrated Endpoint process. This type of counter measures the number of transactions sent to and received from a process running under control of the XPNET process. In addition, you can also configure queue and state user counters for each Integrated Endpoint process. All of these counters are configured using counter attributes in the process definition.

Note: All XPNET performance statistics are generated by the XPNET process.

If desired, Integrated Endpoint processing rates can also be grouped and you can establish maximum and minimum thresholds at which EMS messages are generated. For detailed information on setting up rate, queue, and state user counters for processes running under the XPNET process, grouping counters, and establishing thresholds, refer to the ***NET24 Performance Monitoring Manual***.

Starting a Measurement

Once Measure user counters have been configured, you can start taking measurements of Integrated Endpoint processes. To start a measurement, complete the following steps.

1. Log on to the Tandem and start a Measure command interface session by entering the following command at a TACL prompt.

`TACL > MEASCOM`
2. Add each of the counters to be measured in this session using the ADD COUNTER command. See step 2 under the “Configuration Steps” heading provided earlier in this section for the syntax of this command. If these commands have already been placed in an edit file, simply obey the file.
3. Start a measurement by entering the following command at the MEASCOM prompt, where *\$vol.subvol.file* is the fully qualified name of the file where the measurement results will be recorded. The file should not exist before being named in this command. Measure will automatically create the data file in the specified location with the appropriate data type.

Alternatively, you can also specify the duration of the measurement or the start and end dates and times for the measurement with this command.

```
1+ START MEASUREMENT $vol.subvol.file
```

Once a measurement is started, it will continue until the measurement is explicitly stopped using the STOP MEASUREMENT command or until the time interval specified with the start measurement command expires.

Stopping a Measurement

When a measurement is started with no start and end dates, times, or duration, measurements continue to be taken until the measurement is explicitly stopped. If a start and end date, time, or duration was specified when the measurement was started, the measurement automatically stops when the specified end date and time or duration is reached. To explicitly stop a measurement in progress, complete the following steps.

1. Log on to the Tandem and start a Measure command interface session by entering the following command at a TACL prompt.

```
TACL > MEASCOM
```

2. Stop a measurement that has already been started by entering the following command at the MEASCOM prompt, where *\$vol.subvol.file* is the fully qualified file name where the measurement results are stored.

```
1+ STOP MEASUREMENT $vol.subvol.file
```

The measurement data can be viewed while the measurement is in progress or after the measurement has been stopped. Refer to the “Viewing Measure Results” heading for more information on viewing measurement data.

Viewing Measure Results

This subsection discusses the procedures used to view the results of a measurement.

Viewing Procedures

Once a measurement has been started, you can begin viewing measurement results. The measurement does not have to be stopped to be able to view measurement data. To view the results of a measurement, complete the following steps.

1. Log on to the Tandem and start a Measure command interface session by entering the following command at a TACL prompt.
2. View the results of a measurement that is either currently in progress or has already been stopped by entering the following commands at the MEASCOM prompt, where *\$vol.subvol.file* is the fully qualified file name where the measurement results are stored.

```
TACL > MEASCOM
```

```
1+ ADD MEASUREMENT $vol.subvol.file
2+ LIST USERDEF *, RATE OFF
```

The ADD MEASUREMENT command selects a file for viewing while the LIST USERDEF command displays a report of the measured data. The report can either be displayed on your screen, sent to a printer, or sent to an edit file. The commands shown here display the Measure report on your screen. With the List command, you can also specify a specific time period for which statistics are to be displayed.

These commands display the output for all Measure user counters defined for the entire time period for which the measurement has been in progress and for which Measure data has been collected.

NET24 Performance Monitoring Subsystem

If NET24 Performance Monitoring Subsystem rate, queue, or state counters are established for Integrated Endpoint processes and are added to the Performance Monitoring Subsystem configuration files, you can use the NET24 Performance Monitoring Subsystem facilities to monitor Integrated Endpoint process performance. For detailed information on performance monitoring facilities and procedures, refer to the *NET24 Performance Monitoring Manual*.

ACI Worldwide Inc.

Section 5

Integrated Endpoint Process Configuration

This section describes each of the following files used to configure the Integrated Endpoint process, with the exception of the Destination File (DEST), which is covered in section 3 and appendix A.

- Kernel configuration files, including each of the following:
 - Assign File
 - Assign/Object Relationship File
 - Assign/Process Type Relationship File
 - Extended Memory Table File (EMT)
 - EMT/Object Relationship File
 - EMT/Process Type Relationship File
 - Object File
 - Process/Object Relationship File
 - Process File
 - Process Type File
 - Source Class Map File
 - Server Class Type File
 - Server Class Type/Process Relationship File
- Logical Network Configuration File (LCONF)
- Network Environment File (NEF)

Some files that must be configured do not reside in the integrated transaction services (ITS) environment. These files already reside in the XPNET processing environment. Reference to the appropriate XPNET manual is made whenever appropriate.

Kernel Configuration

The kernel objects and files are required for all ITS processes. The Integrated Endpoint process uses the kernel configuration files and objects just like any other process configured to the ITS environment. Many of these files are provided with default data and do not require any configuration. The following kernel files, however, must be configured during or after installation:

- Process File (PRO)
- Process/Object Relationship File (PORF)
- Server Class Type File (SCT)
- Server Class Type/Process Relationship File (SPRF)

For detailed information on kernel file configuration requirements, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***. This manual also discusses the relationships among kernel files and the recommended order in which the files should be configured.

For detailed configuration requirements for kernel objects used by the Integrated Endpoint process, refer to the ***BASE24 ITS Kernel Objects Reference Manual***.

LCONF Assigns and Params

This subsection lists each of the Logical Network Configuration File (LCONF) assigns and params specific to integrated endpoint objects.

In addition to these assigns and params, the assigns and params used by kernel objects must also be included in your LCONF.

You may also need to add application-specific assigns and params for the Integrated Endpoint process. See the application-specific transaction processing manual for more information on application-specific LCONF assigns and params.

Refer to the *BASE24 Logical Network Configuration Manual* for detailed descriptions of each LCONF assign and param used by kernel and integrated endpoint objects.

Integrated Endpoint LCONF Assigns

The following assigns are specific to integrated endpoint objects.

- CTXEMT-SWAPVOL
- DEST
- DESTEMT

Integrated Endpoint LCONF Params

The following params are specific to integrated endpoint objects.

- CTX-SECND-ALLOC
- CTXEMT-CONTEXT-PAGES

Network Environment File

The Network Environment File (NEF) defines devices, lines, links, nodes, processes, and stations to the XPNET process. All processes and stations running under the XPNET process, including Integrated Endpoint processes, must be defined in the NEF using a network control facility.

For detailed information on the NEF, refer to the *NET24-XPNET Network Control Operations Guide* and the *NET24-XPNET Network Control Command Reference Manual*.

Every station and process that sends messages to the Integrated Endpoint process in the ITS environment requires the declaration of specific attributes used by the Integrated Endpoint process. These parameters are provided or are used to calculate information in the header of XPNET messages sent to the Integrated Endpoint process. Each of these attributes is described below.

Network Overload Attributes

In systems running under XPNET 3.0, the QMT and QMI attributes for a process or station are used to calculate the current network overload percentage by the XPNET process for each message delivered to the Integrated Endpoint process. The XPNET Interface object compares this percentage to an LCONF param value to determine whether the network overload threshold has been exceeded. If so, the XPNET Interface object sets the Network Overload Management Percent field in the network message header to indicate that overload has been detected. The Endpoint Class Handler object can then take whatever actions are appropriate, such as discarding new request messages until the queue overload subsides. For more information on the possible network overload actions performed by the Endpoint Class Handler object, refer to the *BASE24 IEP Objects Reference Manual*.

For detailed information on how the XPNET Interface object detects network overload, refer to the *BASE24 ITS Kernel Configuration Manual*.

Station Attributes

Three additional station attributes are important to configuring the Integrated Endpoint process. Each of these attributes are described below and on the following page. Examples of both ITS and procedural station definitions are provided following these descriptions.

DESTINATION Attribute

The DESTINATION attribute identifies the process that handles all messages received from the station.

For stations used by ITS-based endpoints, the DESTINATION attribute must be set to the symbolic name of the Integrated Endpoint process that contains the endpoint-specific Endpoint Class Handler object for the endpoint the station services. The symbolic name must match the symbolic name configured for the process in the XPNET system and in the Process File (PRO) of the ITS environment. Messages from an ITS-based station are routed directly to an Integrated Endpoint process.

Note: The following information applies only to procedural BASE24 applications.

For stations used by procedural-based endpoints, the DESTINATION attribute should typically be set to the symbolic name of the procedural Device Handler, Host Interface, or Interchange Interface process that handles traffic from the station. The Integrated Endpoint process is typically inserted after the procedural Device Handler or interface process in the transaction path. This is because procedural Device Handler, Host Interface and Interchange Interface processes typically must see each message to handle such information as message sequence numbers and message authentication.

To implement the Integrated Endpoint process into the procedural transaction path after it has been processed by the procedural Device Handler or interface process, the Terminal Data File (TDF) record for a procedural Device Handler process, the Host Configuration File (HCF) for a procedural Host Interface process, or the Interchange Configuration File (ICF) for a procedural Interchange Interface process must be modified to make the Integrated Endpoint process the destination for transactions originating from the procedural-based process.

Note: In addition to the above configuration changes, code changes to existing procedural BASE24 products may also be required to successfully incorporate the Integrated Endpoint process.

SOURCECLASS Attribute

The SOURCECLASS attribute defines the source class for a message. The source class defines the message handler object for a message in the Source Class Map File (SCM) extended memory table. Messages with a particular source class are handled by an endpoint-specific Endpoint Class Handler object. For example, an interactive voice response system (IVR) station could be configured with a source class of IVR. In this case, all messages with a source class of IVR are handled by a Voice Response Unit Endpoint Class Handler object. The source class defined for a station should be descriptive of the endpoint it services. For example, a VISA station could be defined with a source class of VISA, while a host station could be defined with a source class of HOST. The value of the Sourceclass or Class attribute must match a source class entry in the Source Class Map File (SCM).

The source class is also used for message-based routing in the Destination File extended memory table. This table is built from the DEST. For more information on how the source class is used in this file, refer to appendix A.

SERVICE Attribute

The SERVICE attribute identifies the service or services performed by a process or station. The Destination File (DEST) can be configured to route messages to a provider of a service rather than to a specific symbolic destination. For more information on how the SERVICE attribute can be used in this file, refer to appendix A.

ITS Station Example

The following is an example of a definition for a station to an ITS-based endpoint. Note that the DEVICENAME, LINENAME, and DESTINATION attributes are required for a station definition.

```
SET STATION DEVICENAME \SYS1.P2A^NODE.SPARE^DEV1
SET STATION LINENAME \SYS1.P2A^NODE.L2A^DEV1
SET STATION DESTINATION \SYS1.P2A^NODE.P2A^IEP1
SET STATION SOURCECLASS IVR
SET STATION USERDATA 0 0 0 0 0 0 0 0 0 0 0
SET STATION OINTERVAL 0
SET STATION RETRY 0
SET STATION LOGICALACK OFF
SET STATION AUTOPRI 128
SET STATION WARMPRI 128
SET STATION QMT 0
SET STATION QMI 0
```

```
SET STATION SSQCNT 0
ADD STATION \SYS1.P2A^NODE.S2AHOST01
```

Procedural Station Example

The following is an example of a definition for a station that sends messages to a procedural BASE24 application. Messages from this endpoint are first routed to a procedural-based Host Interface process before they are processed by an Integrated Endpoint process. The Host Configuration File (HCF) for the procedural host interface process then would be configured to send the message to the Integrated Endpoint process.

```
SET STATION DEVICENAME \SYS1.P2A^NODE.SPARE^DEV1
SET STATION LINENAME \SYS1.P2A^NODE.L2A^DEV1
SET STATION DESTINATION \SYS1.P1A^NODE.P1A^HISO01
SET STATION SOURCECLASS HOST
SET STATION USERDATA 0 0 0 0 0 0 0 0 0 0 0 0
SET STATION OINTERVAL 0
SET STATION RETRY 0
SET STATION LOGICALACK OFF
SET STATION AUTOPRI 128
SET STATION WARMPRI 128
SET STATION QMT 0
SET STATION QMI 0
SET STATION SSQCNT 0
ADD STATION \SYS1.P2A^NODE.S2AHOST01
```

Process Attributes

Several process attributes, in addition to the network overload attributes described earlier, are important to configuring the Integrated Endpoint process. Each of these attributes are described below and on the following page. Example process definitions for both ITS and procedural processes are provided following these descriptions.

Note: For procedural-based processes such as a Device Handler, Host Interface, or Interchange Interface process, the Integrated Endpoint process is typically specified as the destination process in the configuration file record for the procedural process (that is, the Host Configuration File for a Host Interface process, the Interchange Configuration File for an Interchange Interface process, or the Terminal Data File for a Device Handler process). In addition, code changes to existing procedural BASE24 products may also be required to successfully incorporate the Integrated Endpoint process.

The ADD PROCESS command, as well as the PPD and LIBRARY attributes, are also important to all Integrated Server processes. This command and its attributes are described in the *BASE24 ITS Kernel Configuration Manual*.

SOURCECLASS Attribute

As with stations, the SOURCECLASS attribute defines the source class for a message. The source class defines the message handler object for a message in the Source Class Map File (SCM) extended memory table. Messages with a particular source class are handled by an endpoint-specific Endpoint Class Handler object. For example, a process may be configured with a source class value of IVR. In this case, all messages with a source class value of IVR are handled by a Voice Response Unit Endpoint Class Handler object.

For naming convention purposes, ACI recommends that the SOURCECLASS attributes for ITS processes always be set to the same value as the process type defined for the process in the Process Type File (PTF). The source class is also used for message-based routing in the Destination File extended memory table. This table is built from the DEST. For more information on how the source class is used in this file, refer to appendix A.

SERVICE Attribute

The SERVICE attribute identifies the service or services performed by a process or station. The Destination File (DEST) can be configured to route messages to a provider of a service rather than to a specific symbolic destination. For more information on how the SERVICE attribute can be used in this file, refer to appendix A.

ITS Process Example

The following is an example of a process definition for an Integrated Authorization Server process.

```
SET PROCESS PROGRAM $DATA.OC13OBJ.IASLO
SET PROCESS LIBRARY $SYSTEM.XNLIB.D30LIB
SET PROCESS CPU 1
SET PROCESS BCPU 2
SET PROCESS STARTUP COMMAND
SET PROCESS PPD $C1I1
SET PROCESS QMT 0
SET PROCESS QMI 0
```

```
SET PROCESS SOURCECLASS IAS
SET PROCESS SERVICE TBAUTH
SET PROCESS SAVEABEND ON
SET PROCESS PRIORITY 155
ADD PROCESS \SYS1.P1A^NODE.P1A^IAS01
```

IEP Example

The following is an example of a process definition for an Integrated Endpoint process.

```
SET PROCESS PROGRAM $DATA.OE13OBJ.IEPLO
SET PROCESS LIBRARY $SYSTEM.XNLIB.D30LIB
SET PROCESS CPU 1
SET PROCESS BCPU 2
SET PROCESS STARTUP COMMAND
SET PROCESS PPD $C1I1
SET PROCESS QMT 0
SET PROCESS QMI 0
SET PROCESS SOURCECLASS IEP
SET PROCESS SAVEABEND ON
SET PROCESS PRIORITY 155
ADD PROCESS \SYS1.P1A^NODE.P1A^IEP01
```

Procedural Process Example

The following is an example of a process definition for a procedural BASE24 process.

```
SET PROCESS PROGRAM $DATA.BA53OBJ.HISO
SET PROCESS LIBRARY $SYSTEM.XPNET.SKEL
SET PROCESS CPU 1
SET PROCESS BCPU 0
SET PROCESS STARTUP COMMAND
SET PROCESS PPD $C1H1
SET PROCESS QMT 0
SET PROCESS QMI 0
SET PROCESS SOURCECLASS HOST
SET PROCESS PRIORITY 155
ADD PROCESS \SYS1.P1A^NODE.P1A^HISO1
```

Pathway Configuration File

Due to the architecture of Pathway systems, an Integrated Endpoint process is not required. Thus, Integrated Endpoint processes are not configured to run under the Pathway process. Integrated Endpoint processes can only be configured to run under the XPNET process.

Although the Integrated Endpoint process does not run under the Pathway process, Pathway-based endpoints can still be configured to route messages to an XPNET-based Integrated Endpoint process. In this case, the Path Server process must be inserted between the Pathway-based endpoint and the XPNET process in the transaction path. The Path Server process provides communication services between Pathway- and XPNET-based processes. This process is multithreaded with each thread of control from the Path Server process configured as a station in the XPNET system. The Integrated Endpoint process is configured as the destination process for the Path Server process stations. The source class for a Pathway process endpoint is obtained from the SOURCECLASS attribute values assigned to the Path Server process stations in their process definitions.

Appendix A

Destination File (DEST)

This appendix describes the Destination File (DEST) screen and its fields. The Destination File screen is displayed when you select the INTEGRATED ENDPOINT entry from the BASE24 Virtual Menu.

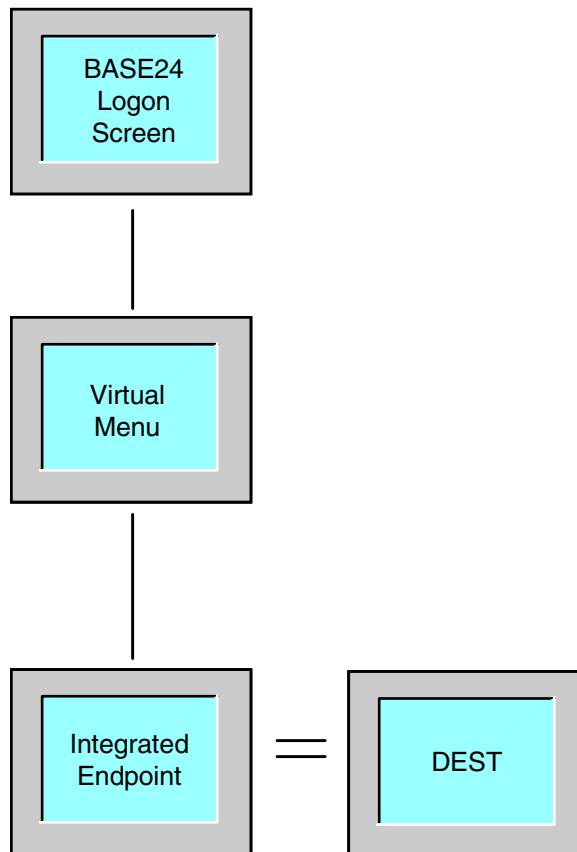
This appendix includes discussions on the following topics:

- Screen access to the DEST screen, including brief descriptions of each screen used to access the DEST screen
- An overview of the DEST screen and its fields
- A sample DEST screen
- Detailed descriptions of each field that appears on the DEST screen

Destination File Screen Access

The Destination File screen is accessed through a series of formatted menus and screens. The menu navigation required to get to the Destination File screen is depicted in the following diagram. Each of the screens in the illustration are described on the following pages.

You can also access the Destination File screen by entering DEST in the FILE DESTINATION field on any BASE24 screen and pressing the **F1** or **F16** key. The **F1** key performs this function on menu screens, while the **F16** key performs this function on file maintenance or process control screens.



BASE24 Logon Screen — The first screen that appears when you log on to the system is the BASE24 Logon screen. On this screen you must enter a valid user name and password. You can also change the screen printer location if desired.

Virtual Menu — The first screen displayed after you successfully log on to the system. Three entries on this screen are used to display integrated transaction services (ITS) configuration and control screens. The KERNEL entry on this screen is used to display ITS kernel configuration and control screens. The CORE entry is used to display application-specific configuration and control screens. The INTEGRATED END POINT entry is used to display the DEST screen. To display the DEST screen, tab the cursor next to the INTEGRATED END POINT entry and press the **F1** key.

Integrated Endpoint — Provides direct access to the Destination File screen.

For a complete description of the BASE24 Logon screen and the Virtual Menu, refer to the ***BASE24 CRT Access Manual***.

Function Keys

The DEST screen uses standard file maintenance function keys. For detailed information on these function keys, refer to the ***BASE24 CRT Access Manual***. You can obtain brief online help for function keys available for the DEST screen by pressing the **F12** key while the DEST screen is displayed.

DEST Screen

The DEST screen provides file maintenance access to the Destination File. This file is used to define the destinations for messages routed based on the source class and message class associated with the given message. One record must exist in this file for each source class, message class, and endpoint destination combination in your system for which message-based routing is used. One or multiple destinations can be configured for each source class and message class combination. In addition, any or all of the Message Class field can be set to wildcard characters. For ITS process destinations, each record contains the object ID of the message handler object to which this type of message will be sent. If the destination is a procedural process, an object ID does not need to be specified.

Corresponding records must already exist in the following files before a record can be added to the DEST.

- Object File (OBJ)
- Source Class Map File (SCM)

The DEST screen is secured under Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

The key to DEST records is a combination of the SOURCE CLASS, MESSAGE CLASS, and DESTINATION fields. A sample of the DEST screen is shown on the following page, followed by descriptions of its fields.

Screen 1

DEST screen 1 allows you to define the message-based routing for your ITS environment. DEST screen 1 is shown below, followed by a description of its function keys and each field on the screen.

```

BASE24-IEP  DESTINATION FILE          SYS1          YY/MM/DD  HH:MM  01 OF 01

                SOURCE CLASS:
                MESSAGE CLASS:
                DESTINATION:

                OBJECT NAME:          (<object description>)
                TLO TO TLO LEXICON FLAG: N
EPCH SERVICES TO OBJECT LEXICON FLAG: N

***** BASE24 *****
NEW PAGE:          FILE DESTINATION:          NEW LOGICAL NETWORK ID:
                  F12-HELP

```

SOURCE CLASS — The source class associated with the process or station from which a message originates. This field must contain the same value as the SOURCECLASS or CLASS attribute defined for the process or station in its NEF definition. A source class provides a means of grouping similar endpoints as one logical entity for message routing.

The SOURCECLASS attribute is only available for systems running under XPNET 2.1 or greater. The value of the SOURCECLASS attribute can be up to four characters in length, while the value of the CLASS attribute can be up to 16 characters in length. If both the SOURCECLASS and CLASS attributes are defined for a process or station in the NEF, only the value of the SOURCECLASS attribute should be configured in the DEST.

This field, in combination with the MESSAGE CLASS and DESTINATION fields, is the primary key to DEST records.

Example: IAS
Field Length: 1–16 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: DEST.PRIKEY.SRC-CLAS

MESSAGE CLASS — A series of characters used to determine how messages are routed. Specific values for these fields depend on the Endpoint Class Handler object that is being used.

This field is divided into four subfields of two characters each when the Destination object searches the DEST. The first two characters of this field define the first subfield, the third and fourth characters define the second subfield, the fifth and sixth characters define the third subfield, and the seventh and eighth characters define the fourth subfield.

The Endpoint Class Handler object is responsible for deriving the message class for each incoming message. Because the Endpoint Class Handler object is written specifically for messages from a particular class of endpoint, each Endpoint Class Handler object can determine the message class in a different manner. For example, one Endpoint Class Handler object could combine fields in the message to form the message class. Another Endpoint Class Handler object could arbitrarily assign a message class based on the source class.

An asterisk (*) denotes a wildcard character in this field.

For more information on how the message class and wildcard character are used in specifying message-based routing, refer to section 3.

Field Length: 8 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: DEST.PRIKEY.MSG-CLAS

DESTINATION —The symbolic name of a process or station, or the name of a service, to which messages from this combination of source class and message class are sent. This name must be defined as a process, station, or service in the XPNET system and cannot be set to a wildcard value. The destination is the entity that processes messages for a particular combination of source and message class.

If the destination is a server class of processes, this field should contain the name of the service assigned to each process in that server class.

If messages from a particular combination of source class and message class are to be sent to multiple destinations simultaneously (broadcast routing), a separate record must be configured for each destination with the same combination of values in the SOURCE CLASS and MESSAGE CLASS fields.

This field, in combination with the SOURCE CLASS and MESSAGE CLASS fields, is the primary key to DEST records.

Field Length: 1–16 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: DEST.PRIKEY.DEST.SYM-DEST

OBJECT NAME — The name of the message handler object within the destination process that receives messages from this combination of source and message class. For ITS destinations, this is the name of the object in the destination process that handles a particular kind of message from a particular source.

Object names are defined in the Object File (OBJ). An object name is required if the TLO TO TLO LEXICON FLAG field is set to the value Y.

Upon entry, the text description of the object is retrieved from the OBJ and is displayed in parentheses to the right of the object name.

This field only applies if the destination process is an ITS process. If the destination is a procedural application process or station, this field should be left blank.

Field Length: 1–4 alphanumeric characters
Required Field: No
Default Value: No default value
Data Name: DEST.OBJ-NAM

<object description> — The description of the object from the OBJ. This field is retrieved from the OBJ after a valid object name is entered in the OBJECT NAME field.

Field Length: System-protected
Data Name: OBJ.OBJ-DESCR

TLO TO TLO LEXICON FLAG — A flag indicating whether the destination expects to receive the Message to Object member function. Valid values are as follows:

- Y = Yes, this destination expects to receive the Message to Object member function.
- N = No, this destination does not expect to receive the Message to Object member function.

This flag should be set to the value Y if the destination process is an ITS process and it expects to receive application messages encapsulated within the Message to Object member function. If the destination process is an ITS process and expects to receive Internal Transaction Data (ITD) context only, this flag should be set to the value N. If the destination process is a procedural process and expects the message to be provided in BASE24 standard internal message format, then this flag must also be set to the value N.

Field Length: 1 alphabetic character
Required Field: No
Default Value: N
Data Name: DEST.TLO-TLO-LEX-FLG

EPCH SERVICES TO OBJECT LEXICON FLAG — A flag indicating whether the destination process requires the application message to be encapsulated within an external message member function in the End Point Class Handler Services to Object lexicon. Valid values are as follows:

- Y = Yes, this destination expects to receive messages encapsulated within an external message member function in the Endpoint Class Handler Services to Object lexicon.
- N = No, this destination does not expect to receive messages encapsulated within an external message member function in the Endpoint Class Handler Services to Object lexicon.

If the message being routed is a member function message sent to an ITS destination process, this flag must be set to the value Y. If the message being routed is formatted as a BASE24 standard internal message or is a message passed through the Integrated Endpoint process from one station to another, this flag must be set to the value N. Therefore, this flag is set to the value Y only if the destination process is an Integrated Server process that expects to receive member function messages or a procedural destination process expects to receive additional information in the message. This field is generally set to the value Y if the TLO TO TLO LEXICON FLAG field is also set to the value Y.

For procedural destinations, this flag should be set to the value Y only if the destination expects to receive additional information in the message that the application requires. If the procedural destination has not been constructed to expect any additional information in the message, this flag should always be set to N.

Field Length:	1 alphabetic character
Required Field:	No
Default Value:	N
Data Name:	DEST.EPCS-TO-OBJ-LEX-FLG

Glossary

This glossary explains terms commonly used when discussing message-based routing.

Broadcast routing — A method of routing messages to multiple destinations in parallel. Broadcasting allows a message from an endpoint to be sent to an unlimited number of destinations, simultaneously.

Chained routing — A method of routing messages sequentially from one endpoint to another endpoint. Each endpoint does some processing of the message and sends it back to the originating Integrated Endpoint process for further routing. At a minimum, either the message class or the source class must differ from one endpoint to the next.

Endpoint class — A set of transaction sources or destinations that share a common message format. An endpoint class is defined for each external interface, such as a host system, interchange, or device type.

Endpoint Class Handler object — An object that contains all of the functions and procedures required to process the message types for a particular endpoint class. Whereas the endpoint class is a set of transaction types, the Endpoint Class Handler object is a set of procedures for processing the transaction types defined by the endpoint class. The Endpoint Class Handler object can interpret the format of messages received from an endpoint class. The Endpoint Class Handler object is capable of selecting the appropriate fields from a message to derive the message class. Any time a new endpoint class is added with new message formats (for example, a new device or interchange), a new Endpoint Class Handler object must be created.

Integrated Endpoint process — An object-oriented satellite process to the XPNET process that routes messages, maintains transaction context, and manipulates message counts for messages received from an endpoint. An Integrated Endpoint process always contains an endpoint-specific Endpoint Class Handler object.

Integrated Server process — Any server process running in the ITS environment that is composed from objects. An Integrated Server process is composed of the following kernel objects as well as application-specific objects: Application Network Services, Class, Event, Trace, and XPNET Interface. An Integrated Server process provides business transaction services, such as card management, authorization, external interface management, or transaction logging.

Message class — A set of message types. In general, a message class groups similar messages. Examples of a message class are network management messages and automated teller machine (ATM) withdrawal transactions. The message class for a message is determined by the Endpoint Class Handler object that uses the message content.

Object — A combination of data and procedural statements that act upon that data. An object acts on member function messages and performs appropriate actions.

Object-oriented design — A software development methodology utilizing small, reusable, easily managed units of code called objects. Objects are used as building blocks to create processes.

Pass-through routing — A method of routing messages received from an endpoint station to another station, without entering an application. Messages are simply passed through the Tandem system.

Server class — A set of identical server processes whose number can vary with changes in workload, thereby providing application scalability. By assigning an entire group of identical server processes to a server class, a client (sending application) does not have to open a specific server process; rather, it opens the class and is assigned a link to an available server process within that class. A separate service is typically associated with each server class for the routing of messages.

Simple routing — A method of routing messages from an endpoint to an online processing destination.

Source class — A means of grouping similar endpoints as one logical entity for message routing. The source class is defined for each process and station endpoint in the Network Environment File (NEF).

Subsystem identifier (SSID) — A number identifying an event generator to the Event Management Service (EMS). Event suppression is activated by EMS subsystem IDs (SSIDs) and event numbers configured in the Event Suppression File (ESF).

Symbolic destination routing — A method of routing where a message is sent to a single symbolic destination based on the message contents and the logic coded directly into the Endpoint Class Handler object for the process. Symbolic destination routing of a message can be used with or without message-based routing.

XPNET process — The main controlling process of an XPNET network. Most processes and stations interface with each other through the XPNET process.

Wildcard routing — The means by which the message class can be manipulated with wildcard characters to create generalized or specific routing scenarios.

ACI Worldwide Inc.

Index

A

ADD PROCESS command, [5-8](#)

B

BASE24 Logon screen, [A-3](#)

Broadcast routing
 example of, [3-17](#)
 introduction, [1-9](#)

C

Chained routing
 example of, [3-22](#)
 introduction, [1-9](#)

Combined broadcast and chained routing
 example of, [3-26](#)
 introduction, [1-10](#)

Configuration files
 core, [1-14](#)
 DEST, [1-13](#), [A-5](#)
 IFCF, [1-14](#)
 kernel, [1-13](#), [5-2](#)
 LCONF, [1-13](#), [5-3](#)
 Measure, [1-13](#)
 NEF, [1-13](#), [5-4](#)
 procedural, [1-14](#)

Context extended memory table, [1-15](#)

D

DEST
 see Destination File (DEST) screen

DESTINATION attribute, [5-5](#)

DESTINATION field, [A-7](#)

Destination File (DEST) screen
 DESTINATION field, [A-7](#)
 EPCH SERVICES TO OBJECT LEXICON FLAG field, [A-9](#)
 field descriptions, [A-6](#)
 function keys, [A-4](#)
 MESSAGE CLASS field, [A-7](#)
 Object description field, [A-8](#)
 OBJECT NAME field, [A-8](#)
 screen 1, [A-6](#)
 SOURCE CLASS field, [A-6](#)
 TLO TO TLO LEXICON FLAG field, [A-9](#)

Destination File extended memory table, [1-15](#)

Destination File field data names
 EPCS-TO-OBJ-LEX-FLG, [A-10](#)
 OBJ-DESCR, [A-8](#)
 OBJ-NAM, [A-8](#)
 PRIKEY.DEST.SYM-DEST, [A-8](#)
 TLO-TLO-LEX-FLG, [A-9](#)

Destination process, [2-4](#)

E

Endpoint, definition of, [1-1](#)

EPCH SERVICES TO OBJECT LEXICON FLAG field, [A-9](#)

Event messages, [1-18](#)

Event suppression configuration, [1-20](#)

EVENTCX file, [1-18](#)

Extended memory tables, [1-15](#)

F

Flat segments, [1-22](#)

H

HCF
 see Host Configuration File (HCF)

High PIN support, [1-21](#)

Host Configuration File (HCF), [1-14](#), [5-5](#)

I

ICF
 see Interchange Configuration File (ICF)

IFCF
 see ITS Interface Configuration File (IFCF)

Integrated Endpoint process
 configuration procedures, [1-16](#)
 configuration requirements, [2-5](#)
 description of, [1-3](#)
 ITS integration, [2-2](#)
 multiple processes, [1-5](#)
 number of processes, [1-4](#)
 partitioned processes, [1-6](#)
 pathway integration, [2-12](#)
 procedural integration, [2-4](#), [2-9](#)

Integrated Endpoint process *continued*

- processing, [2-4](#)
- single process, [1-4](#)

Integrated Endpoint process configuration

- broadcast routing, [3-17](#)
- chained routing, [3-22](#)
- combined broadcast and chained routing, [3-26](#)
- core files, [1-14](#), [4-5](#)
- Destination File (DEST), [1-13](#), [1-16](#), [A-1](#)
- file introduction, [1-13](#)
- kernel files, [1-13](#), [1-16](#)
- LCONF, [1-13](#), [1-16](#), [5-3](#)
- Measure, [1-13](#), [4-4](#)
- message-based routing, [3-1](#)
- NEF, [1-13](#), [1-16](#), [5-4](#)
- pass-through routing, [3-13](#)
- procedural configuration files, [1-14](#)
- procedural files, [1-17](#)
- simple routing, [3-10](#)
- symbolic destination routing, [1-11](#)
- wildcard routing, [3-30](#)

Interchange Configuration File (ICF), [5-5](#)Interface extended memory table, [1-15](#)ITS Interface Configuration File (IFCF), [1-14](#), [3-5](#)**K**Kernel configuration requirements, [5-2](#)**L**Large-memory model, [1-22](#)Logging on to the system, [A-3](#)Logical Network Configuration File (LCONF), [1-13](#)

Logical Network Configuration File (LCONF) assigns

- CTXEMT-SWAPVOL, [5-3](#)
- DEST, [5-3](#)
- DESTEMT, [5-3](#)

Logical Network Configuration File (LCONF) params

- CTXEMT-CONTEXT-PAGES, [5-3](#)
- CTX-SECND-ALLOC, [5-3](#)

Logon screen, [A-3](#)**M**

Measure

- introduction, [4-1](#)
- usage of, [1-12](#)
- user counters, [1-12](#), [4-2](#), [4-4](#)

Measure configuration file, [1-13](#)

Measure counter configuration

- Endpoint Class Handler object, [4-2](#)
- Endpoint Class Handler Services object, [4-2](#)
- introduction, [4-1](#)
- procedures, [4-4](#)
- rates, [4-3](#), [4-6](#)

Measure counter reporting

- introduction, [4-1](#)
- NET24 Performance Monitoring subsystem, [4-9](#)
- starting a measurement, [4-7](#)
- stopping a measurement, [4-8](#)
- viewing results, [4-9](#)

Memory models

- large, [1-22](#)
- small, [1-22](#)
- toggle, [1-22](#)

MESSAGE CLASS field, [A-7](#)Message class wildcard flag, [3-4](#)

Message-based routing

- broadcast, [1-9](#), [3-17](#)
- chained, [1-9](#), [3-22](#)
- combined broadcast and chained routing, [1-10](#), [3-26](#)
- introduction, [1-9](#)
- pass-through, [1-9](#), [3-13](#)
- simple, [1-9](#), [3-10](#)
- wildcard routing, [3-30](#)

N

NEF

- see* Network Environment File (NEF)

NET24 Performance Monitoring subsystem

- configuration of, [4-6](#)
- viewing Measure results, [4-9](#)

Network Environment File (NEF)

- configuration requirements, [5-4](#)
- introduction, [1-13](#)

Nucleus process terminology change, [xvi](#)**O**Object description field, [A-8](#)OBJECT NAME field, [A-8](#)**P**Partitioned process, [1-6](#)

Pass-through routing

- example of, [3-13](#)
- introduction, [1-9](#)

Pathway Configuration file, [5-10](#)

PORF

- see* Process/Object Relationship File (PORF)

PRO

- see* Process File (PRO)

Process attributes

- introduction, [5-7](#)
- LIBRARY, [5-8](#)
- PPD, [5-8](#)
- QMI, [5-4](#)
- QMT, [5-4](#)
- SERVICE, [5-8](#)
- SOURCECLASS, [5-8](#)

Process File (PRO), [5-2](#)

Process/Object Relationship File (PORF), [5-2](#)

Q

QMI attribute, [5-4](#)

QMT attribute, [5-4](#)

S

Screens

accessing, [A-2](#)

logging on, [A-3](#)

standard function keys, [A-4](#)

SCT

see Server Class Type File (SCT)

Server Class Type File (SCT), [5-2](#)

Server Class Type/Process Relationship File (SPRF), [5-2](#)

SERVICE attribute

process, [5-8](#)

stations, [5-6](#)

Simple routing

example of, [3-10](#)

introduction, [1-9](#)

Small-memory model, [1-22](#)

SOURCE CLASS field, [A-6](#)

SOURCECLASS attribute, [5-6](#)

SPRF

see Server Class Type/Process Relationship File (SPRF)

Station attributes

DESTINATION, [5-5](#)

introduction, [5-5](#)

QMI, [5-4](#)

QMT, [5-4](#)

SERVICE, [5-6](#)

SOURCECLASS, [5-6](#)

Symbolic destination routing

description of, [1-11](#)

T

TDF

see Terminal Data File (TDF)

Template file names of ACI source files, [1-18](#)

Terminal Data File (TDF), [5-5](#)

TLO TO TLO LEXICON FLAG field, [A-9](#)

V

VRU Configuration Data (VCD) screen, [4-4](#)

W

Wildcard routing

examples of, [3-30](#)

introduction, [3-30](#)

X

XPNET feature support

high PIN, [1-21](#)

large-memory model, [1-22](#)

XPNET process definition, [xvi](#)

ACI Worldwide Inc.